

Project Vitriol – NEA Implementation

Vitriol.Data

Vitriol.Data\Db

Vitriol.Data\Db\DbPaths.cs

```
namespace Vitriol.Data.Db;
```

```
public static class DbPaths
```

```
{
```

```
    public static string DefaultDbFile(string projectRoot)
```

```
        => Path.Combine(projectRoot, "data", "vitriol.db");
```

```
}
```

Vitriol.Data\Db\SqlConnectionFactory.cs

```
using Microsoft.Data.Sqlite;
```

```
namespace Vitriol.Data.Db;
```

```
public sealed class SqlConnectionFactory(string connectionString)
```

```
{
```

```
    public SqlConnection Create()
```

```
        => new(connectionString);
```

```
    public static SqlConnectionFactory ForFile(string dbFilePath)
```

```
{
```

```
    Directory.CreateDirectory(Path.GetDirectoryName(dbFilePath)!);
```

```
    var cs = new SqlConnectionStringBuilder
```

```
{
```

```
        DataSource = dbFilePath,
```

```

        Mode = SqliteOpenMode.ReadWriteCreate,
        Cache = SqliteCacheMode.Shared
    }.ToString();

    return new(cs);
}
}

Vitriol.Data\Migrations
Vitriol.Data\Migrations\SqliteConnectionFactory.cs
using Microsoft.Data.Sqlite;

namespace Vitriol.Data.Migrations;

public sealed class MigrationRunner
{
    public void EnsureMigrated(SqliteConnection conn)
    {
        // 1) migration history table
        using (var cmd = conn.CreateCommand())
        {
            cmd.CommandText =
                """
                CREATE TABLE IF NOT EXISTS __migrations (
                    id INTEGER PRIMARY KEY,
                    name TEXT NOT NULL,
                    applied_utc TEXT NOT NULL
                );
                """,

```

```
cmd.ExecuteNonQuery();  
}
```

```
// 2) Apply migrations in order
```

```
ApplyIfMissing(conn, 1, "001_init_species", ""
```

```
CREATE TABLE IF NOT EXISTS pokemon_species (
```

```
    species_id INTEGER PRIMARY KEY,
```

```
    name TEXT NOT NULL UNIQUE,
```

```
    primary_type TEXT NOT NULL,
```

```
    secondary_type TEXT NULL,
```

```
    base_hp INTEGER NOT NULL,
```

```
    base_atk INTEGER NOT NULL,
```

```
    base_def INTEGER NOT NULL,
```

```
    base_spa INTEGER NOT NULL,
```

```
    base_spd INTEGER NOT NULL,
```

```
    base_spe INTEGER NOT NULL
```

```
);
```

```
""");
```

```
}
```

```
private static void ApplyIfMissing(SqliteConnection conn, long id, string name, string  
sql)
```

```
{
```

```
    using var check = conn.CreateCommand();
```

```
    check.CommandText = "SELECT 1 FROM __migrations WHERE id = $id LIMIT 1;";
```

```
    check.Parameters.AddWithValue("$id", id);
```

```
    var exists = check.ExecuteScalar() is not null;
```

```

    if (exists) return;

    using var tx = conn.BeginTransaction();

    using (var cmd = conn.CreateCommand())
    {
        cmd.Transaction = tx;

        cmd.CommandText = sql;

        cmd.ExecuteNonQuery();
    }

    using (var insert = conn.CreateCommand())
    {
        insert.Transaction = tx;

        insert.CommandText =

            "INSERT INTO __migrations(id, name, applied_utc) VALUES($id, $name, $utc);";

        insert.Parameters.AddWithValue("$id", id);

        insert.Parameters.AddWithValue("$name", name);

        insert.Parameters.AddWithValue("$utc", DateTime.UtcNow.ToString("O"));

        insert.ExecuteNonQuery();
    }

    tx.Commit();
}
}

Vitriol.Data\Repositories
Vitriol.Data\Repositories\PokemonSpeciesRepository.cs
using Microsoft.Data.Sqlite;

```

```
using Vitriol.Shared.Pokemon;
```

```
namespace Vitriol.Data.Repositories;
```

```
public sealed class PokemonSpeciesRepository
```

```
{
```

```
    public long Insert(SqliteConnection conn, PokemonSpeciesDto dto)
```

```
    {
```

```
        using var cmd = conn.CreateCommand();
```

```
        cmd.CommandText =
```

```
            ""
```

```
            INSERT INTO pokemon_species
```

```
                (name, primary_type, secondary_type, base_hp, base_atk, base_def, base_spa,  
base_spd, base_spe)
```

```
            VALUES
```

```
                ($name, $pt, $st, $hp, $atk, $def, $spa, $spd, $spe);
```

```
            SELECT last_insert_rowid();
```

```
            ""
```

```
        cmd.Parameters.AddWithValue("$name", dto.Name);
```

```
        cmd.Parameters.AddWithValue("$pt", dto.PrimaryType);
```

```
        cmd.Parameters.AddWithValue("$st", (object?)dto.SecondaryType ??  
DBNull.Value);
```

```
        cmd.Parameters.AddWithValue("$hp", dto.BaseHP);
```

```
        cmd.Parameters.AddWithValue("$atk", dto.BaseAttack);
```

```
        cmd.Parameters.AddWithValue("$def", dto.BaseDefense);
```

```
        cmd.Parameters.AddWithValue("$spa", dto.BaseSpAttack);
```

```
        cmd.Parameters.AddWithValue("$spd", dto.BaseSpDefense);
```

```
        cmd.Parameters.AddWithValue("$spe", dto.BaseSpeed);
```

```

        return (long)cmd.ExecuteScalar()!;
    }

    public PokemonSpeciesDto? GetByName(SqliteConnection conn, string name)
    {
        using var cmd = conn.CreateCommand();

        cmd.CommandText =
            """
            SELECT species_id, name, primary_type, secondary_type, base_hp, base_atk,
            base_def, base_spa, base_spd, base_spe

            FROM pokemon_species

            WHERE name = $name

            LIMIT 1;
            """,
            cmd.Parameters.AddWithValue("$name", name);

        using var r = cmd.ExecuteReader();

        if (!r.Read()) return null;

        return new PokemonSpeciesDto(
            SpeciesId: r.GetInt64(0),
            Name: r.GetString(1),
            PrimaryType: r.GetString(2),
            SecondaryType: r.IsDBNull(3) ? null : r.GetString(3),
            BaseHP: r.GetInt32(4),
            BaseAttack: r.GetInt32(5),
            BaseDefense: r.GetInt32(6),

```

```
        BaseSpAttack: r.GetInt32(7),  
        BaseSpDefense: r.GetInt32(8),  
        BaseSpeed: r.GetInt32(9)  
    );  
}  
}
```

Vitriol.Data\Class1.cs

```
namespace Vitriol.Data;
```

```
public class Class1
```

```
{
```

```
}
```

Vitriol.Data\Vitriol.Data.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
<ItemGroup>
```

```
  <ProjectReference Include="..\Vitriol.Shared\Vitriol.Shared.csproj" />
```

```
</ItemGroup>
```

```
<ItemGroup>
```

```
  <PackageReference Include="Microsoft.Data.Sqlite" Version="10.0.1" />
```

```
</ItemGroup>
```

```
<PropertyGroup>
```

```
  <TargetFramework>net9.0</TargetFramework>
```

```
  <ImplicitUsings>enable</ImplicitUsings>
```

```
  <Nullable>enable</Nullable>
```

```
</PropertyGroup>
```

```
</Project>
```

Vitriol.Editor

Vitriol.Editor\App.xaml

```
<Application x:Class="Vitriol.Editor.App"
```

```
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
```

```
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
```

```
    ShutdownMode="OnMainWindowClose">
```

```
    <Application.Resources>
```

```
    </Application.Resources>
```

```
</Application>
```

Vitriol.Editor\App.xaml.cs

```
using System;
```

```
using System.Windows;
```

```
using System.Windows.Threading;
```

```
namespace Vitriol.Editor;
```

```
public partial class App : Application
```

```
{
```

```
    public App()
```

```
    {
```

```
        // Catch UI thread exceptions
```

```
        DispatcherUnhandledException += (_, e) =>
```

```
        {
```

```
            MessageBox.Show(e.Exception.ToString(), "Vitriol Editor crashed (UI thread)");
```



```
e.Handled = true;  
Shutdown(-1);  
};
```

```
// Catch non-UI exceptions  
AppDomain.CurrentDomain.UnhandledException += (_, e) =>  
{  
    MessageBox.Show(e.ExceptionObject?.ToString() ?? "Unknown error",  
        "Vitriol Editor crashed (AppDomain)");  
};  
}
```

```
protected override void OnStartup(StartupEventArgs e)  
{  
    base.OnStartup(e);  
  
    try  
    {  
        // Force-create and show MainWindow so StartupUri issues can't silently exit the  
app  
        var window = new MainWindow();  
        MainWindow = window;  
        window.Show();  
    }  
    catch (Exception ex)  
    {  
        MessageBox.Show(ex.ToString(), "Vitriol Editor failed during startup");  
        Shutdown(-1);  
    }  
}
```

```
    }  
  }  
}
```

Vitriol.Editor\AssemblyInfo.cs

```
using System.Windows;
```

```
[assembly:ThemeInfo(  
    ResourceDictionaryLocation.None,    //where theme specific resource  
    dictionaries are located  
  
    //(used if a resource is not found in the page,  
    // or application resource dictionaries)  
  
    ResourceDictionaryLocation.SourceAssembly //where the generic resource  
    dictionary is located  
  
    //(used if a resource is not found in the page,  
    // app, or any theme specific resource dictionaries)  
)]
```

Vitriol.Editor\MainWindow.xaml

```
<Window x:Class="Vitriol.Editor.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Vitriol Editor"  
    Height="720"  
    Width="1280"  
    WindowState="Maximized"  
    WindowStartupLocation="CenterScreen">  
  
    <Grid Margin="12">  
        <Grid.RowDefinitions>  
            <RowDefinition Height="Auto"/>
```

```

        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*/>
    </Grid.RowDefinitions>

    <WrapPanel Orientation="Horizontal" Margin="0,0,0,8"
VerticalAlignment="Center">

        <Button Width="160" Height="32" Margin="0,0,8,0" Click="OnInitClick">

            Init Project DB

        </Button>

        <Border BorderBrush="#444" BorderThickness="1" CornerRadius="4"
Padding="8,6" Margin="0,0,8,0">

            <StackPanel Orientation="Horizontal">

                <TextBlock Text="Map Size:" VerticalAlignment="Center" Margin="0,0,8,0"/>

                <TextBlock Text="W" VerticalAlignment="Center" Margin="0,0,4,0"/>

                <TextBox Name="MapWidthBox" Width="70" Height="24" Text="300"
Margin="0,0,10,0"/>

                <TextBlock Text="H" VerticalAlignment="Center" Margin="0,0,4,0"/>

                <TextBox Name="MapHeightBox" Width="70" Height="24" Text="200"
Margin="0,0,10,0"/>

            </StackPanel>

            <Button Width="180" Height="28" Click="OnCreateSampleMapClick">

                Create Sample Map

            </Button>

            <!-- NEW: Create New Map -->

            <Button Width="160" Height="28" Margin="8,0,0,0"
Click="OnCreateNewMapClick">

```

Create New Map

</Button>

<Button Width="140" Height="28" Margin="8,0,0,0"
Click="OnLoadMapClick">

Load Map

</Button>

<!-- NEW: Save Map -->

<Button Width="120" Height="28" Margin="8,0,0,0" Click="OnSaveMapClick">

Save

</Button>

</StackPanel>

</Border>

<Separator Width="12" Opacity="0"/>

<Button Width="90" Height="32" Margin="0,0,6,0"
Click="OnZoomOutClick">Zoom -</Button>

<Button Width="90" Height="32" Margin="0,0,6,0"
Click="OnZoomResetClick">Reset</Button>

<Button Width="90" Height="32" Click="OnZoomInClick">Zoom +</Button>

<TextBlock Margin="16,6,0,0"

Foreground="#666"

VerticalAlignment="Center"

Text="Playtest: F5 toggle, WASD/Arrows move, E/Enter interact, Esc exit"/>

</WrapPanel>

```
<DockPanel Grid.Row="1" Margin="0,0,0,8">
```

```
  <TextBlock Name="StatusText"
```

```
    FontSize="14"
```

```
    DockPanel.Dock="Left"
```

```
    TextTrimming="CharacterEllipsis"/>
```

```
  <TextBlock Name="ZoomText"
```

```
    FontSize="14"
```

```
    DockPanel.Dock="Right"
```

```
    Margin="12,0,0,0"/>
```

```
  <TextBlock Name="MouseText"
```

```
    FontSize="14"
```

```
    DockPanel.Dock="Right"/>
```

```
</DockPanel>
```

```
<Grid Grid.Row="2">
```

```
  <Grid.ColumnDefinitions>
```

```
    <ColumnDefinition Width="360"/>
```

```
    <ColumnDefinition Width="12"/>
```

```
    <ColumnDefinition Width="*/>
```

```
  </Grid.ColumnDefinitions>
```

```
  <Border Grid.Column="0" BorderBrush="Gray" BorderThickness="1"
  CornerRadius="4" Padding="10">
```

```
    <ScrollViewer VerticalScrollBarVisibility="Auto"
```

```
      HorizontalScrollBarVisibility="Disabled"
```

```
IsManipulationEnabled="True"
ScrollViewer.PanningMode="VerticalOnly"
ScrollViewer.CanContentScroll="False"
Stylus.IsPressAndHoldEnabled="False"
Stylus.IsFlicksEnabled="False">
```

```
<StackPanel>
```

```
    <TextBlock FontSize="16" FontWeight="SemiBold" Text="Tools"
Margin="0,0,0,8"/>
```

```
    <StackPanel Orientation="Horizontal" Margin="0,0,0,10">
```

```
        <TextBlock Text="Edit Mode:" VerticalAlignment="Center"
Margin="0,0,8,0"/>
```

```
        <RadioButton Name="ModeTiles"
```

```
            Content="Tiles"
```

```
            IsChecked="True"
```

```
            Margin="0,0,10,0"
```

```
            Checked="OnModeChanged"/>
```

```
        <RadioButton Name="ModeCollision"
```

```
            Content="Collision"
```

```
            Margin="0,0,10,0"
```

```
            Checked="OnModeChanged"/>
```

```
        <RadioButton Name="ModeEvents"
```

```
            Content="Events"
```

```
            Checked="OnModeChanged"/>
```

```
    </StackPanel>
```

```
    <CheckBox Name="ShowCollisionOverlay"
```

```
Content="Show collision overlay"
IsChecked="False"
Checked="OnOverlayChanged"
Unchecked="OnOverlayChanged"
Margin="0,0,0,6"/>
```

```
<CheckBox Name="ShowEventsOverlay"
    Content="Show events overlay"
    IsChecked="True"
    Checked="OnOverlayChanged"
    Unchecked="OnOverlayChanged"
    Margin="0,0,0,14"/>
```

```
<Separator Margin="0,0,0,12"/>
```

```
<!-- Tile palette + tileset management -->
```

```
<TextBlock FontSize="16" FontWeight="SemiBold" Text="Tile Palette"
Margin="0,0,0,8"/>
```

```
<!-- Tileset controls (button + checkboxes live here) -->
```

```
<Border BorderBrush="#444" BorderThickness="1" CornerRadius="4"
Padding="8" Margin="0,0,0,10">
```

```
<StackPanel>
```

```
<Button Height="28"
```

```
Margin="0,0,0,8"
```

```
Click="OnLoadTilesetsClick">
```

```
Load Tileset PNG(s)...
```

```
</Button>
```

```
<TextBlock FontWeight="SemiBold" Text="Loaded Tilesets"
Margin="0,0,0,6"/>
```

```
<ScrollViewer VerticalScrollBarVisibility="Auto"
    HorizontalScrollBarVisibility="Disabled"
    Height="120"
    IsManipulationEnabled="True"
    ScrollViewer.PanningMode="VerticalOnly"
    ScrollViewer.CanContentScroll="False"
    Stylus.IsPressAndHoldEnabled="False"
    Stylus.IsFlicksEnabled="False">
    <StackPanel Name="TilesetListPanel"/>
</ScrollViewer>
```

```
<TextBlock Foreground="#666"
    Margin="0,8,0,0"
    TextWrapping="Wrap"
    Text="Tip: Uncheck a tileset to unload it (keeps indices stable, but
tiles render blank until re-enabled)."/>
</StackPanel>
</Border>
```

```
<!-- Palette thumbnails -->
<Border BorderBrush="#444" BorderThickness="1" CornerRadius="4"
Padding="6" Margin="0,0,0,14">
    <ScrollViewer VerticalScrollBarVisibility="Auto"
        HorizontalScrollBarVisibility="Disabled"
        Height="220"
        IsManipulationEnabled="True"
```



```

        ScrollViewer.PanningMode="VerticalOnly"
        ScrollViewer.CanContentScroll="False"
        Stylus.IsPressAndHoldEnabled="False"
        Stylus.IsFlicksEnabled="False">
        <WrapPanel Name="TilePalettePanel"/>
    </ScrollViewer>
</Border>

<Separator Margin="0,0,0,12"/>

    <TextBlock FontSize="16" FontWeight="SemiBold" Text="Event Editor"
Margin="0,0,0,8"/>

    <TextBlock Name="EventSelectedText" Foreground="#666"
Margin="0,0,0,8"/>

    <StackPanel Orientation="Horizontal" Margin="0,0,0,8">

        <TextBlock Width="70" Text="Name:" VerticalAlignment="Center"/>

        <TextBox Name="EventNameBox" Width="240"
LostFocus="OnEventNameLostFocus"/>

    </StackPanel>

    <StackPanel Orientation="Horizontal" Margin="0,0,0,8">

        <TextBlock Width="70" Text="Type:" VerticalAlignment="Center"/>

        <ComboBox Name="EventTypeBox" Width="240"
SelectionChanged="OnEventTypeChanged"/>

    </StackPanel>

    <Button Name="DeleteEventButton"

        Height="28"

```

Width="120"

Margin="0,0,0,12"

Click="OnDeleteEventClick">

Delete

</Button>

<StackPanel Name="WarpFieldsPanel" Visibility="Collapsed"
Margin="0,0,0,12">

<TextBlock FontWeight="SemiBold" Text="Warp Settings"
Margin="0,0,0,6"/>

<StackPanel Orientation="Horizontal" Margin="0,0,0,6">

<TextBlock Width="70" Text="destMap:" VerticalAlignment="Center"/>

<ComboBox Name="WarpDestMapBox"

Width="240"

IsEditable="True"

DropDownOpened="OnWarpDestMapDropDownOpened"

LostFocus="OnWarpFieldsLostFocus"/>

</StackPanel>

<StackPanel Orientation="Horizontal" Margin="0,0,0,6">

<TextBlock Width="70" Text="destX:" VerticalAlignment="Center"/>

<TextBox Name="WarpDestXBox" Width="70"
LostFocus="OnWarpFieldsLostFocus"/>

<TextBlock Width="30"/>

<TextBlock Width="70" Text="destY:" VerticalAlignment="Center"/>

<TextBox Name="WarpDestYBox" Width="70"
LostFocus="OnWarpFieldsLostFocus"/>

</StackPanel>

</StackPanel>

```
<Border BorderBrush="#444" BorderThickness="1" CornerRadius="4"
Padding="8" Margin="0,0,0,12">
```

```
<Grid>
```

```
<Grid.RowDefinitions>
```

```
<RowDefinition Height="Auto"/>
```

```
<RowDefinition Height="*/>
```

```
</Grid.RowDefinitions>
```

```
<TextBlock Grid.Row="0"
```

```
FontSize="14"
```

```
FontWeight="SemiBold"
```

```
Text="Parameters"
```

```
Margin="0,0,0,8"/>
```

```
<Grid Grid.Row="1">
```

```
<Grid.RowDefinitions>
```

```
<RowDefinition Height="Auto"/>
```

```
<RowDefinition Height="*/>
```

```
</Grid.RowDefinitions>
```

```
<Grid Grid.Row="0" Margin="0,0,0,8">
```

```
<Grid.ColumnDefinitions>
```

```
<ColumnDefinition Width="*/>
```

```
<ColumnDefinition Width="Auto"/>
```

```
</Grid.ColumnDefinitions>
```

```
<Grid Grid.Column="0" Margin="0,0,8,0">
```

```
<Grid.ColumnDefinitions>
```

```

        <ColumnDefinition Width="Auto"/>
        <ColumnDefinition Width="*/>
        <ColumnDefinition Width="Auto"/>
        <ColumnDefinition Width="*/>
    </Grid.ColumnDefinitions>

    <TextBlock Grid.Column="0" Text="Key:"
VerticalAlignment="Center" Margin="0,0,6,0"/>

        <TextBox Grid.Column="1" Name="ParamKeyBox" Height="26"
MinWidth="90" Margin="0,0,12,0"/>

    <TextBlock Grid.Column="2" Text="Value:"
VerticalAlignment="Center" Margin="0,0,6,0"/>

        <TextBox Grid.Column="3" Name="ParamValueBox" Height="26"
MinWidth="90"/>

    </Grid>

    <StackPanel Grid.Column="1" Orientation="Vertical">

        <Button Width="76" Height="26" Click="OnAddParamClick"
Margin="0,0,0,6">Add</Button>

        <Button Width="76" Height="26"
Click="OnDeleteParamClick">Delete</Button>

    </StackPanel>

</Grid>

<ListBox Grid.Row="1"
    Name="ParamsList"
    SelectionChanged="OnParamSelectionChanged"
    MinHeight="110"
    IsManipulationEnabled="True"

```

```

        ScrollViewer.PanningMode="VerticalOnly"
        ScrollViewer.CanContentScroll="False"
        Stylus.IsPressAndHoldEnabled="False"
        Stylus.IsFlicksEnabled="False"/>
    </Grid>
</Grid>
</Border>

<Separator Margin="0,0,0,12"/>

<TextBlock FontSize="16" FontWeight="SemiBold" Text="Event Script"
Margin="0,0,0,8"/>

<ListBox Name="EventCommandList"
    Height="160"
    SelectionChanged="OnCommandSelectionChanged"
    Margin="0,0,0,8"
    IsManipulationEnabled="True"
    ScrollViewer.PanningMode="VerticalOnly"
    ScrollViewer.CanContentScroll="False"
    Stylus.IsPressAndHoldEnabled="False"
    Stylus.IsFlicksEnabled="False"/>

<WrapPanel Margin="0,0,0,10">
    <Button Width="90" Height="28" Margin="0,0,6,6"
Click="OnAddShowTextClick">+ ShowText</Button>

    <Button Width="90" Height="28" Margin="0,0,6,6"
Click="OnAddSetFlagClick">+ SetFlag</Button>

```

```
        <Button Width="80" Height="28" Margin="0,0,6,6"
Click="OnDeleteCommandClick">Delete</Button>
```

```
        <Button Width="60" Height="28" Margin="0,0,6,6"
Click="OnMoveCommandUpClick">Up</Button>
```

```
        <Button Width="60" Height="28" Margin="0,0,6,6"
Click="OnMoveCommandDownClick">Down</Button>
```

```
    </WrapPanel>
```

```
    <Border BorderBrush="#444" BorderThickness="1" CornerRadius="4"
Padding="8">
```

```
        <StackPanel>
```

```
            <TextBlock Name="CommandEditorHeader"
```

```
                FontWeight="SemiBold"
```

```
                Text="No command selected"
```

```
                Margin="0,0,0,8"/>
```

```
        <StackPanel Name="ShowTextEditorPanel" Visibility="Collapsed">
```

```
            <TextBlock Text="Text:" Margin="0,0,0,4"/>
```

```
            <TextBox Name="CmdTextBox"
```

```
                Height="70"
```

```
                AcceptsReturn="True"
```

```
                TextWrapping="Wrap"
```

```
                LostFocus="OnCommandFieldsLostFocus"/>
```

```
        </StackPanel>
```

```
    <StackPanel Name="SetFlagEditorPanel" Visibility="Collapsed">
```

```
        <StackPanel Orientation="Horizontal" Margin="0,0,0,6">
```

```
            <TextBlock Width="70" Text="Flag:" VerticalAlignment="Center"/>
```

```

        <TextBox Name="CmdFlagNameBox" Width="240"
LostFocus="OnCommandFieldsLostFocus"/>

    </StackPanel>

    <StackPanel Orientation="Horizontal">

        <TextBlock Width="70" Text="Value:" VerticalAlignment="Center"/>

        <TextBox Name="CmdFlagValueBox" Width="240"
LostFocus="OnCommandFieldsLostFocus"/>

    </StackPanel>

</StackPanel>

</Border>

</StackPanel>

</ScrollViewer>

</Border>

<Border Grid.Column="2" BorderBrush="Gray" BorderThickness="1"
CornerRadius="4">

    <ScrollViewer Name="MapScroll"

        HorizontalScrollBarVisibility="Auto"

        VerticalScrollBarVisibility="Auto"

        ScrollChanged="OnScrollChanged"

        PreviewMouseWheel="OnScrollViewerMouseWheel">

        <Canvas Name="MapCanvas"

            Background="#111111"

            MouseMove="OnMapCanvasMouseMove"

            MouseLeave="OnMapCanvasMouseLeave"

```

```
        MouseLeftButtonDown="OnMapCanvasMouseLeftButtonDown"
        MouseRightButtonDown="OnMapCanvasMouseRightButtonDown"/>
    </ScrollViewer>
</Border>
</Grid>
</Grid>
</Window>
```

Vitriol.Editor\MainWindow.xaml.cs

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Threading;
using Microsoft.Win32;
using Vitriol.Data.Db;
using Vitriol.Data.Migrations;
using Vitriol.Engine.Maps;
using Vitriol.Shared.Maps;
using Vitriol.Shared.Validation;
using IOPath = System.IO.Path;

namespace Vitriol.Editor;
```



```

public partial class MainWindow : Window
{
    // -----

    // Tileset caching + runtime compilation

    // -----

    private readonly ConcurrentDictionary<string, BitmapSource> _tilesetBitmapCache =
new(StringComparer.OrdinalIgnoreCase);

    private readonly ConcurrentDictionary<string, ConcurrentDictionary<int,
CroppedBitmap>> _tileCropCache = new(StringComparer.OrdinalIgnoreCase);


    // Stores computed metadata so global indices remain stable even if a tileset is
unloaded.

    private readonly ConcurrentDictionary<string, TilesetMeta> _tilesetMetaCache =
new(StringComparer.OrdinalIgnoreCase);


    // Tracks whether a tileset is currently enabled (checkbox state).

    private readonly ConcurrentDictionary<string, bool> _tilesetLoadedState =
new(StringComparer.OrdinalIgnoreCase);


    private readonly object _tilesetCompileLock = new();
    private List<TilesetRuntime> _compiledTilesets = new();
    private bool _tilesetsDirty = true;


    // -----

    // Zoom state

    // -----

    private double _zoom = 1.0;

    private const double ZoomMin = 0.25;

    private const double ZoomMax = 6.0;

```

```

private const double ZoomStep = 1.15;

// -----
// Viewport renderer state
// -----
private MapDto? _currentMap;
private Image? _viewportImage;
private readonly DispatcherTimer _renderDebounce;

// -----
// Selection + undo/redo
// -----
private int _selectedTileIndex = 0;
private Border? _selectedPaletteBorder;

private readonly Stack<IEditCommand> _undo = new();
private readonly Stack<IEditCommand> _redo = new();

private string? _currentMapPath;

// -----
// Paint state
// -----
private bool _isPainting = false;
private int _lastPaintIdx = -1;

// -----
// Hover state (used for keyboard actions like Delete)

```

```

// -----
private int _hoverTileX = -1;
private int _hoverTileY = -1;

// -----
// Cursor tile selection (for arrow-key cell move)
// -----
private int _cursorTileX = -1;
private int _cursorTileY = -1;

// -----
// Event drag state (smooth overlay; no full redraw spam)
// -----
private bool _draggingEvent = false;
private string? _dragEventId = null;

// Mouse world position (MapCanvas coords)
private double _dragMouseWorldX = 0;
private double _dragMouseWorldY = 0;

// The snapped/clamped tile preview used for commit
private int _dragPreviewX = -1;
private int _dragPreviewY = -1;

// Overlay visuals (type-shaped ghost)
private System.Windows.Shapes.Shape? _dragGhostShape;
private System.Windows.Shapes.Shape? _dragGhostOutlineShape;

```

```

// Swap Hint Overlay (arrow-key swap feedback)

private System.Windows.Shapes.Rectangle? _swapHintFrom;
private System.Windows.Shapes.Rectangle? _swapHintTo;
private readonly DispatcherTimer _swapHintTimer = new();

// -----

// Drag autoscroll (while moving events)
// -----

private readonly DispatcherTimer _dragAutoScrollTimer = new();
private double _dragLastViewportX = 0;
private double _dragLastViewportY = 0;

private const double DragAutoScrollMargin = 48;
private const double DragAutoScrollSpeed = 26;
private const int DragAutoScrollTickMs = 16;

private void DeleteAtCurrentMode()
{
    var map = _currentMap;
    if (map is null) return;

    // Needs a valid hovered tile
    if (_hoverTileX < 0 || _hoverTileY < 0) return;
    if (_hoverTileX >= map.Width || _hoverTileY >= map.Height) return;

    int idx = _hoverTileY * map.Width + _hoverTileX;

    // Don't allow destructive edits during playtest

```

```
if (_playtestActive)
```

```
{
```

```
    StatusText.Text = "Playtest is active (F5 to stop). Editing is disabled.";
```

```
    return;
```

```
}
```

```
if (_mode == EditMode.Tiles)
```

```
{
```

```
    var bg = GetBackgroundLayer();
```

```
    if (bg is null) return;
```

```
    int oldValue = bg.Tiles[idx];
```

```
    int newValue = -1; // erase tile
```

```
    if (oldValue == newValue) return;
```

```
    ExecuteCommand(new PaintIntCellCommand(this, LayerKind.Background, idx,
oldValue, newValue));
```

```
    return;
```

```
}
```

```
if (_mode == EditMode.Collision)
```

```
{
```

```
    var col = GetCollisionLayer();
```

```
    if (col is null) return;
```

```
    int oldValue = col.Tiles[idx];
```

```
    int newValue = 0; // clear collision
```

```

        if (oldValue == newValue) return;

        ExecuteCommand(new PaintIntCellCommand(this, col.Kind, idx, oldValue,
newValue));

        return;
    }

    if (_mode == EditMode.Events)
    {
        var ev = GetEventAt(_hoverTileX, _hoverTileY) ?? GetSelectedEvent();

        if (ev is null) return;

        ExecuteCommand(new DeleteEventCommand(this, ev));

        if (_selectedEventId == ev.EventId)
        {
            _selectedEventId = null;
            _selectedCommandIndex = -1;
            UpdateEventPanel(null);
            RefreshCommandList(null);
            UpdateCommandEditor(null);
        }

        RequestViewportRender();

        return;
    }
}

// -----

```

```

// Ctrl + D Duplicates Event
// -----
private MapEventDto CloneEventTo(MapEventDto src, int x, int y)
{
    var newParams = src.Parameters is null
        ? new Dictionary<string, string>()
        : new Dictionary<string, string>(src.Parameters);

    var newCmds = (src.Commands ?? Array.Empty<EventCommandDto>())
        .Select(c => new EventCommandDto(
            (c.Command ?? "").Trim(),
            c.Args is null ? new Dictionary<string, string>() : new Dictionary<string,
string>(c.Args)))
        .ToArray();

    var name = string.IsNullOrEmpty(src.Name) ? "Event" : src.Name.Trim();

    return new MapEventDto(
        EventId: Guid.NewGuid().ToString("N"),
        Name: name + " Copy",
        Type: (src.Type ?? "Trigger").Trim(),
        X: x,
        Y: y,
        Parameters: newParams,
        Commands: newCmds
    );
}

```

```

private bool TryFindFreeTileNear(int startX, int startY, out int outX, out int outY)
{
    outX = startX;
    outY = startY;

    var map = _currentMap;
    if (map is null) return false;

    // Spiral-ish search radius
    const int maxR = 25;

    bool IsFree(int x, int y) => GetEventAt(x, y) is null;

    if (startX >= 0 && startY >= 0 && startX < map.Width && startY < map.Height &&
        IsFree(startX, startY))
        return true;

    for (int r = 1; r <= maxR; r++)
    {
        int minX = Math.Max(0, startX - r);
        int maxX = Math.Min(map.Width - 1, startX + r);
        int minY = Math.Max(0, startY - r);
        int maxY = Math.Min(map.Height - 1, startY + r);

        for (int x = minX; x <= maxX; x++)
        {
            if (IsFree(x, minY)) { outX = x; outY = minY; return true; }
            if (IsFree(x, maxY)) { outX = x; outY = maxY; return true; }
        }
    }
}

```



```

    }

    for (int y = minY; y <= maxY; y++)
    {
        if (IsFree(minX, y)) { outX = minX; outY = y; return true; }
        if (IsFree(maxX, y)) { outX = maxX; outY = y; return true; }
    }
}

return false;
}

```

```

// -----

// Editor modes
// -----

private enum EditMode { Tiles, Collision, Events }
private EditMode _mode = EditMode.Tiles;

private bool _showCollisionOverlay = false;
private bool _showEventsOverlay = true;

// -----

// Event selection
// -----

private string? _selectedEventId;

// -----

// Command selection (event script commands)

```

```

// -----
private int _selectedCommandIndex = -1;
private bool _updatingCommandUi = false;

// -----
// Playtest state
// -----
private bool _playtestActive = false;
private int _playerX = 0;
private int _playerY = 0;
private readonly Dictionary<string, string> _playtestFlags =
new(StringComparer.OrdinalIgnoreCase);

// -----
// Project folders
// -----
private string ProjectRoot => IOPath.Combine(Directory.GetCurrentDirectory(),
"Vitriol.Projects");
private string MapsFolder => IOPath.Combine(ProjectRoot, "maps");
private string PngsFolder => IOPath.Combine(ProjectRoot, "assets", "tilesets");

// Cached list of available map IDs (from the maps folder) for Warp destMap dropdown.
private List<string> _availableMapIds = new();

private static string? TryMapIdFromFileName(string fileName)
{
    // Supports both "*.vitmap.json" (default) and legacy "*.json"
    if (string.IsNullOrEmpty(fileName)) return null;

```

```

var name = IOPath.GetFileName(fileName);

if (name.EndsWith(MapFormat.DefaultExtension,
StringComparison.OrdinalIgnoreCase))
    name = name[..^MapFormat.DefaultExtension.Length];
else if (name.EndsWith(".json", StringComparison.OrdinalIgnoreCase))
    name = name[..^5];

name = (name ?? "").Trim();
return name.Length == 0 ? null : name;
}

private void RefreshAvailableMaps(bool preserveCurrentSelection = true)
{
    try
    {
        Directory.CreateDirectory(MapsFolder);

        var files = Directory.EnumerateFiles(MapsFolder, "*",
SearchOption.TopDirectoryOnly)
            .Where(f =>
                f.EndsWith(MapFormat.DefaultExtension,
StringComparison.OrdinalIgnoreCase) ||
                f.EndsWith(".json", StringComparison.OrdinalIgnoreCase))
            .ToList();

        var ids = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
        foreach (var f in files)

```

```

{
    var id = TryMapIdFromFileName(f);
    if (id is null) continue;
    ids.Add(id);
}

var list = ids.OrderBy(x => x, StringComparer.OrdinalIgnoreCase).ToList();
_availableMapIds = list;

// If the Warp dest map UI exists (ComboBox in XAML), bind it.
if (WarpDestMapBox is ComboBox cb)
{
    var current = preserveCurrentSelection ? GetWarpDestMapUiValue() : null;

WarpDestMapBox.ItemsSource = null;
WarpDestMapBox.ItemsSource = _availableMapIds;

if (!string.IsNullOrEmpty(current))
{
    if (!_availableMapIds.Contains(current, StringComparer.OrdinalIgnoreCase))
    {
        _availableMapIds = _availableMapIds
            .Concat(new[] { current })
            .Distinct(StringComparer.OrdinalIgnoreCase)
            .OrderBy(x => x, StringComparer.OrdinalIgnoreCase)
            .ToList();

WarpDestMapBox.ItemsSource = null;

```

```

        WarpDestMapBox.ItemsSource = _availableMapIds;
    }

    WarpDestMapBox.SelectedItem = _availableMapIds.FirstOrDefault(x =>
        x.Equals(current, StringComparison.OrdinalIgnoreCase));
}
else
{
    WarpDestMapBox.SelectedItem = null;
}

    }
}
catch
{
    // Swallow folder IO errors; warp dropdown will just be empty.
    _availableMapIds = new();
}
}

private string GetWarpDestMapUiValue()
{
    // WarpDestMapBox is a ComboBox in XAML
    var sel = WarpDestMapBox.SelectedItem as string;
    if (!string.IsNullOrEmpty(sel)) return sel.Trim();

    // If editable, or if user typed
    return (WarpDestMapBox.Text ?? "").Trim();
}

```

```
}
```

```
private void SetWarpDestMapUiValue(string? value)
```

```
{
```

```
    value = (value ?? "").Trim();
```

```
    if (string.IsNullOrEmpty(value))
```

```
    {
```

```
        WarpDestMapBox.SelectedItem = null;
```

```
        WarpDestMapBox.Text = "";
```

```
        return;
```

```
    }
```

```
    // Ensure list contains the value so selection works.
```

```
    if (_availableMapIds.Count == 0 || !_availableMapIds.Contains(value,  
StringComparer.OrdinalIgnoreCase))
```

```
    {
```

```
        _availableMapIds = _availableMapIds
```

```
            .Concat(new[] { value })
```

```
            .Distinct(StringComparer.OrdinalIgnoreCase)
```

```
            .OrderBy(x => x, StringComparer.OrdinalIgnoreCase)
```

```
            .ToList();
```

```
        WarpDestMapBox.ItemsSource = null;
```

```
        WarpDestMapBox.ItemsSource = _availableMapIds;
```

```
    }
```

```
    WarpDestMapBox.SelectedItem = _availableMapIds.FirstOrDefault(x =>
```

```

        x.Equals(value, StringComparison.OrdinalIgnoreCase));

// Keeps display sane even if not selectable
WarpDestMapBox.Text = value;
}

// XAML handler (ComboBox.DropDownOpened): refresh available maps right before
// showing the list.
private void OnWarpDestMapDropDownOpened(object sender, System.EventArgs e)
{
    RefreshAvailableMaps(preserveCurrentSelection: true);
}

// Manual save gating (new maps must be saved once before autosave)
private bool _manualSavePerformed = false;

private static bool IsSampleMapPath(string? path)
{
    if (string.IsNullOrEmpty(path)) return false;
    return path.IndexOf("test_map_01", StringComparison.OrdinalIgnoreCase) >= 0;
}

private bool IsSampleMapLoaded()
{
    if (_currentMap is null) return false;
    if (_currentMap.MapId.Equals("test_map_01",
StringComparison.OrdinalIgnoreCase)) return true;
    return IsSampleMapPath(_currentMapPath);
}

```

```

private static string MakeSafeMapId(string raw)
{
    raw = (raw ?? "").Trim();

    if (raw.Length == 0) return "new_map";

    var chars = raw.Select(ch =>
    {
        if (char.IsLetterOrDigit(ch)) return char.ToLowerInvariant(ch);
        if (ch == '_') return '_';
        if (ch == ' ' || ch == '-') return '_';
        return '\0';
    }).Where(c => c != '\0').ToArray();

    var s = new string(chars);
    while (s.Contains("__", StringComparison.Ordinal)) s = s.Replace("__", "_");
    s = s.Trim('_');

    if (s.Length == 0) s = "new_map";
    return s;
}

public MainWindow()
{
    InitializeComponent();

    RefreshAvailableMaps(preserveCurrentSelection: false);
}

```



```
StatusText.Text = "Editor started. Database not initialised.";
```

```
MouseText.Text = "";
```

```
EventTypeBox.ItemsSource = new[] { "Trigger", "NPC", "Warp" };
```

```
EventTypeBox.SelectedIndex = 0;
```

```
ApplyZoom();
```

```
_renderDebounce = new DispatcherTimer { Interval =  
    TimeSpan.FromMilliseconds(30) };
```

```
_renderDebounce.Tick += (_, _) =>
```

```
{
```

```
    _renderDebounce.Stop();
```

```
    RenderViewport();
```

```
};
```

```
MapScroll.SizeChanged += (_, _) => RequestViewportRender();
```

```
PreviewKeyDown += OnMainWindowPreviewKeyDown;
```

```
MapCanvas.MouseLeftButtonUp += OnMapCanvasMouseLeftButtonUp;
```

```
MapCanvas.MouseRightButtonUp += (_, _) => StopPainting();
```

```
MapCanvas.MouseLeave += OnMapCanvasMouseLeave;
```

```
EnableTouchScrollForListBox(ParamsList);
```

```
EnableTouchScrollForListBox(EventCommandList);
```

```

UpdateEventPanel(null);

RefreshCommandList(null);

UpdateCommandEditor(null);


RefreshTilesetPanel();


_dragAutoScrollTimer.Interval =
TimeSpan.FromMilliseconds(DragAutoScrollTickMs);

_dragAutoScrollTimer.Tick += (_, _) => TickDragAutoScroll();
}


// -----
// Tileset runtime structures
// -----

private sealed record TilesetMeta(
    int PixelWidth,
    int PixelHeight,
    int TileWidth,
    int TileHeight,
    int Margin,
    int Spacing,
    int TilesAcross,
    int TilesDown,
    int TileCount
);


private sealed class TilesetRuntime
{

```

```

public required TilesetRefDto Ref { get; init; }
public required string FullPath { get; init; }
public required string Key { get; init; }
public required int StartIndexGlobal { get; init; }
public required int TileCount { get; init; }
public required int TilesAcross { get; init; }
public required int TileW { get; init; }
public required int TileH { get; init; }
public required int Margin { get; init; }
public required int Spacing { get; init; }
public required bool IsLoaded { get; init; }
public BitmapSource? Bitmap { get; init; }
public ConcurrentDictionary<int, CroppedBitmap>? CropCache { get; init; }
}

```

```

private void MarkTilesetsDirty()
{
    lock (_tilesetCompileLock)
        _tilesetsDirty = true;
}

```

```

private static string NormalizeTilesetKey(string relativeOrAbsolutePath) =>
    (relativeOrAbsolutePath ?? "").Replace("\\", '/').Trim();

```

```

private bool IsTilesetLoaded(string tilesetKey)
{
    tilesetKey = NormalizeTilesetKey(tilesetKey);
    if (tilesetKey.Length == 0) return false;
}

```

```

        if (!_tilesetLoadedState.TryGetValue(tilesetKey, out var loaded))
            return loaded;

        _tilesetLoadedState[tilesetKey] = true;
        return true;
    }

    private void SetTilesetLoaded(string tilesetKey, bool loaded, bool
clearCachesOnUnload = true)
    {
        tilesetKey = NormalizeTilesetKey(tilesetKey);
        if (tilesetKey.Length == 0) return;

        _tilesetLoadedState[tilesetKey] = loaded;

        if (!loaded && clearCachesOnUnload)
        {
            var full = ResolveTilesetFullPathFromKey(tilesetKey);
            if (full is not null)
            {
                _tilesetBitmapCache.TryRemove(full, out _);
                _tileCropCache.TryRemove(full, out _);
            }
        }

        MarkTilesetsDirty();
        RefreshTilesetPanel();
    }

```

```
BuildTilePalette();  
RequestViewportRender();  
}
```

```
private string? ResolveTilesetFullPathFromKey(string tilesetKey)  
{  
    if (tilesetKey.Length == 0) return null;  
  
    if (IOPath.IsPathRooted(tilesetKey))  
        return tilesetKey;  
  
    return IOPath.Combine(ProjectRoot, tilesetKey.Replace('/', '\\'));  
}
```

```
private void EnsureTilesetsCompiled()  
{  
    var map = _currentMap;  
    if (map is null)  
    {  
        lock (_tilesetCompileLock)  
        {  
            _compiledTilesets = new List<TilesetRuntime>();  
            _tilesetsDirty = false;  
        }  
        return;  
    }  
}
```

```
lock (_tilesetCompileLock)
```

```

{
    if (!_tilesetsDirty) return;

    var compiled = new List<TilesetRuntime>();
    int runningStart = 0;

    var tilesets = map.Tilesets ?? Array.Empty<TilesetRefDto>();
    foreach (var ts in tilesets)
    {
        var key = NormalizeTilesetKey(ts.ImagePath ?? "");
        if (key.Length == 0) continue;

        bool loaded = IsTilesetLoaded(key);

        var fullPath = ResolveTilesetFullPathFromKey(key);
        if (fullPath is null) continue;

        BitmapSource? bmp = null;
        if (loaded && File.Exists(fullPath))
        {
            bmp = _tilesetBitmapCache.GetOrAdd(fullPath, LoadBitmap);
        }

        TilesetMeta meta;

        if (bmp is not null)
        {
            meta = ComputeMeta(ts, bmp);
            _tilesetMetaCache[fullPath] = meta;
        }
    }
}

```

```

else if (_tilesetMetaCache.TryGetValue(fullPath, out var metaCached))
{
    meta = metaCached;
}
else
{
    if (File.Exists(fullPath))
    {
        try
        {
            var temp = _tilesetBitmapCache.GetOrAdd(fullPath, LoadBitmap);
            meta = ComputeMeta(ts, temp);
            _tilesetMetaCache[fullPath] = meta;

            if (!loaded)
                _tilesetBitmapCache.TryRemove(fullPath, out _);
        }
        catch
        {
            meta = new TilesetMeta(0, 0,
                Math.Max(1, ts.TileWidth),
                Math.Max(1, ts.TileHeight),
                Math.Max(0, ts.Margin),
                Math.Max(0, ts.Spacing),
                0, 0, 0);
        }
    }
    else

```

```

{
    meta = new TilesetMeta(0, 0,
        Math.Max(1, ts.TileWidth),
        Math.Max(1, ts.TileHeight),
        Math.Max(0, ts.Margin),
        Math.Max(0, ts.Spacing),
        0, 0, 0);
}
}

```

```

ConcurrentDictionary<int, CroppedBitmap>? cropCache = null;
if (loaded && bmp is not null && meta.TileCount > 0)
    cropCache = _tileCropCache.GetOrAdd(fullPath, _ => new
ConcurrentDictionary<int, CroppedBitmap>());

```

```

compiled.Add(new TilesetRuntime
{
    Ref = ts,
    FullPath = fullPath,
    Key = key,
    StartIndexGlobal = runningStart,
    TileCount = meta.TileCount,
    TilesAcross = meta.TilesAcross,
    TileW = meta.TileWidth,
    TileH = meta.TileHeight,
    Margin = meta.Margin,
    Spacing = meta.Spacing,
    IsLoaded = loaded && bmp is not null,

```



```

        Bitmap = bmp,
        CropCache = cropCache
    });

    runningStart += meta.TileCount;
}

_compiledTilesets = compiled;
_tilesetsDirty = false;
}
}

private static TilesetMeta ComputeMeta(TilesetRefDto ts, BitmapSource bmp)
{
    int tileW = Math.Max(1, ts.TileWidth);
    int tileH = Math.Max(1, ts.TileHeight);
    int margin = Math.Max(0, ts.Margin);
    int spacing = Math.Max(0, ts.Spacing);

    int stepX = tileW + spacing;
    int stepY = tileH + spacing;

    int tilesAcross = stepX > 0 ? (bmp.PixelWidth - margin) / stepX : 0;
    int tilesDown = stepY > 0 ? (bmp.PixelHeight - margin) / stepY : 0;

    tilesAcross = Math.Max(0, tilesAcross);
    tilesDown = Math.Max(0, tilesDown);
}

```

```

int tileCount = tilesAcross * tilesDown;

return new TilesetMeta(
    PixelWidth: bmp.PixelWidth,
    PixelHeight: bmp.PixelHeight,
    TileWidth: tileW,
    TileHeight: tileH,
    Margin: margin,
    Spacing: spacing,
    TilesAcross: tilesAcross,
    TilesDown: tilesDown,
    TileCount: tileCount
);
}

private TilesetRuntime? FindRuntimeForGlobalTile(int globalTileIndex)
{
    EnsureTilesetsCompiled();
    if (globalTileIndex < 0) return null;

    foreach (var ts in _compiledTilesets)
    {
        if (globalTileIndex >= ts.StartIndexGlobal &&
            globalTileIndex < ts.StartIndexGlobal + ts.TileCount)
            return ts;
    }

    return null;
}

```

```
}
```

```
private static int GetLocalIndex(TilesetRuntime ts, int globalTileIndex) =>
```

```
    globalTileIndex - ts.StartIndexGlobal;
```

```
// -----
```

```
// Tileset UI panel (checkboxes)
```

```
// -----
```

```
private void RefreshTilesetPanel()
```

```
{
```

```
    if (TilesetListPanel is null) return;
```

```
    TilesetListPanel.Children.Clear();
```

```
    if (_currentMap is null)
```

```
    {
```

```
        TilesetListPanel.Children.Add(new TextBlock { Text = "No map loaded.",  
Foreground = Brushes.Gray });
```

```
        return;
```

```
    }
```

```
    var tilesets = _currentMap.Tilesets ?? Array.Empty<TilesetRefDto>();
```

```
    var visible = tilesets.Where(t => !string.IsNullOrEmpty(t.ImagePath)).ToList();
```

```
    if (visible.Count == 0)
```

```
    {
```

```
        TilesetListPanel.Children.Add(new TextBlock
```

```
        {
```

```

        Text = "No tilesets referenced by the current map.",
        Foreground = Brushes.Gray
    });
    return;
}

foreach (var ts in visible)
{
    var key = NormalizeTilesetKey(ts.ImagePath ?? "");
    if (key.Length == 0) continue;

    var row = new DockPanel { Margin = new Thickness(0, 0, 0, 6) };

    var cb = new CheckBox
    {
        IsChecked = IsTilesetLoaded(key),
        VerticalAlignment = VerticalAlignment.Center,
        Margin = new Thickness(0, 0, 8, 0)
    };

    cb.Checked += (_, _) => SetTilesetLoaded(key, true);
    cb.Unchecked += (_, _) => SetTilesetLoaded(key, false);

    DockPanel.SetDock(cb, Dock.Left);
    row.Children.Add(cb);

    row.Children.Add(new TextBlock
    {

```

```

        Text = $"{ts.TilesetId} ({key})",
        TextWrapping = TextWrapping.Wrap
    });

    TilesetListPanel.Children.Add(row);
}
}

// -----
// Touch scrolling helper
// -----

private static void EnableTouchScrollForListBox(ListBox listBox)
{
    if (listBox == null) return;

    listBox.IsManipulationEnabled = true;

    listBox.PreviewTouchMove += (_, e) => { e.Handled = true; };
}

// -----
// Database initialization
// -----

private void OnInitClick(object sender, RoutedEventArgs e)
{
    Directory.CreateDirectory(IOPath.Combine(ProjectRoot, "data"));

    var dbFile = DbPaths.DefaultDbFile(ProjectRoot);
    var factory = SqlConnectionFactory.ForFile(dbFile);

```

```

using var conn = factory.Create();

conn.Open();

new MigrationRunner().EnsureMigrated(conn);

StatusText.Text = $"Database ready: {dbFile}";
}

// -----
// Map creation/loading
// -----

private void OnCreateSampleMapClick(object sender, RoutedEventArgs e)
{
    if (!TryReadMapSize(out int w, out int h))
        return;

    Directory.CreateDirectory(MapsFolder);

    var map = CreateSampleMap(w, h);

    var errors = MapValidator.Validate(map);
    if (errors.Count > 0)
    {
        StatusText.Text = "Map validation failed: " +
            string.Join("; ", errors.Select(er => $"{er.Field}: {er.Message}"));

        return;
    }
}

```

```
    var path = IOPath.Combine(MapsFolder, map.MapId +  
MapFormat.DefaultExtension);
```

```
    MapSerializer.SaveToFile(path, map);
```

```
    LoadMapFromPath(path, "Saved and loaded sample map");
```

```
}
```

```
private void OnLoadMapClick(object sender, RoutedEventArgs e)
```

```
{
```

```
    Directory.CreateDirectory(MapsFolder);
```

```
    var dlg = new OpenFileDialog
```

```
{
```

```
        Title = "Open Vitriol Map",
```

```
        Filter = "Vitriol Map (*.vitmap.json)|*.vitmap.json|JSON (*.json)|*.json|All files  
(**)|**",
```

```
        InitialDirectory = MapsFolder,
```

```
        CheckFileExists = true,
```

```
        Multiselect = false
```

```
};
```

```
if (dlg.ShowDialog(this) != true)
```

```
    return;
```

```
    LoadMapFromPath(dlg.FileName, "Loaded map");
```

```
}
```

```
private void LoadMapFromPath(string path, string statusPrefix)
```

```
{
```

```

if (!File.Exists(path))
{
    StatusText.Text = $"Map file not found: {path}";
    return;
}

try
{
    _currentMapPath = path;
    _currentMap = MapSerializer.LoadFromFile(path);
    _manualSavePerformed = !IsSampleMapPath(path);

    if (_currentMap.Events is null)
        _currentMap = _currentMap with { Events = Array.Empty<MapEventDto>() };

    var fixedEvents = _currentMap.Events
        .Select(ev =>
        {
            var cmds = ev.Commands ?? Array.Empty<EventCommandDto>();
            var pars = ev.Parameters ?? new Dictionary<string, string>();
            if (ev.Commands == cmds && ev.Parameters == pars) return ev;
            return ev with { Commands = cmds, Parameters = pars };
        })
        .ToList();

    _currentMap = _currentMap with { Events = fixedEvents };

    _selectedEventId = null;

```



```
_selectedCommandIndex = -1;

UpdateEventPanel(null);

RefreshCommandList(null);

UpdateCommandEditor(null);


EnsureCollisionLayer();


_undo.Clear();
_redo.Clear();


var tilesets = _currentMap.Tilesets ?? Array.Empty<TilesetRefDto>();
foreach (var ts in tilesets)
{
    var key = NormalizeTilesetKey(ts.ImagePath ?? "");
    if (key.Length == 0) continue;
    _tilesetLoadedState.TryAdd(key, true);
}


MarkTilesetsDirty();


EnsureViewportImageExists();

RefreshTilesetPanel();

BuildTilePalette();


MapScroll.ScrollToHorizontalOffset(0);

MapScroll.ScrollToVerticalOffset(0);


RefreshAvailableMaps(preserveCurrentSelection: false);
```

```
        StatusText.Text = $"{statusPrefix}: {path}";  
        RequestViewportRender();  
    }  
    catch (Exception ex)  
    {  
        StatusText.Text = $"Failed to load map: {ex.Message}";  
    }  
}
```

```
private bool TryReadMapSize(out int w, out int h)  
{  
    w = 0;  
    h = 0;  
  
    if (!int.TryParse(MapWidthBox.Text?.Trim(), out w) ||  
        !int.TryParse(MapHeightBox.Text?.Trim(), out h))  
    {  
        StatusText.Text = "Width/Height must be whole numbers.";  
        return false;  
    }  
  
    if (w < 1 || h < 1)  
    {  
        StatusText.Text = "Width/Height must be at least 1.";  
        return false;  
    }  
}
```

```

    if (w > 2000 || h > 2000)
    {
        StatusText.Text = "Width/Height too large (max 2000 each).";
        return false;
    }

    return true;
}

private static MapDto CreateNewBlankMap(string mapId, string name, int w, int h)
{
    const int tileSize = 32;

    var tiles = Enumerable.Repeat(-1, w * h).ToArray();
    var collision = new int[w * h];

    var map = new MapDto(
        FormatVersion: MapFormat.CurrentVersion,
        MapId: mapId,
        Name: name,
        Width: w,
        Height: h,
        TileSize: tileSize,
        Tilesets: new[]
        {
            new TilesetRefDto(
                TilesetId: "default_tileset",
                ImagePath: "",

```

```

        TileWidth: 32,
        TileHeight: 32,
        Margin: 0,
        Spacing: 0
    )
},
Layers: new[]
{
    new TileLayerDto("bg", "Background", LayerKind.Background, true, tiles),
    new TileLayerDto("col", "Collision", LayerKind.Collision, true, collision)
}
);

return map with { Events = Array.Empty<MapEventDto>() };
}

```

```

private static MapDto CreateSampleMap(int w, int h)
{
    const int tileSize = 32;

    var tiles = Enumerable.Repeat(-1, w * h).ToArray();
    var collision = new int[w * h];

    var map = new MapDto(
        FormatVersion: MapFormat.CurrentVersion,
        MapId: "test_map_01",
        Name: "Test Map 01",
        Width: w,

```

```

    Height: h,
    TileSize: tileSize,
    Tilesets: new[]
    {
        new TilesetRefDto(
            TilesetId: "default_tileset",
            ImagePath: "",
            TileWidth: 32,
            TileHeight: 32,
            Margin: 0,
            Spacing: 0
        )
    },
    Layers: new[]
    {
        new TileLayerDto("bg", "Background", LayerKind.Background, true, tiles),
        new TileLayerDto("col", "Collision", LayerKind.Collision, true, collision)
    }
);

return map with { Events = Array.Empty<MapEventDto>() };
}

// -----
// Collision layer compatibility
// -----

private void EnsureCollisionLayer()
{

```

```

var map = _currentMap;

if (map is null) return;

bool hasCollision =
    map.Layers.Any(l =>
        l.Kind == LayerKind.Collision ||
        l.LayerId == "col" ||
        l.Name.Equals("Collision", StringComparison.OrdinalIgnoreCase));

if (hasCollision) return;

var col = new TileLayerDto(
    LayerId: "col",
    Name: "Collision",
    Kind: LayerKind.Collision,
    Visible: true,
    Tiles: new int[map.Width * map.Height]
);

var newLayers = map.Layers.Concat(new[] { col }).ToArray();
_currentMap = map with { Layers = newLayers };

PersistMap();
}

private TileLayerDto? GetBackgroundLayer() =>
    _currentMap?.Layers.FirstOrDefault(l => l.Kind == LayerKind.Background);

```

```

private TileLayerDto? GetCollisionLayer() =>
    _currentMap?.Layers.FirstOrDefault(l => l.Kind == LayerKind.Collision || l.LayerId ==
"col");

// -----
// Tool UI events
// -----

private void OnModeChanged(object sender, RoutedEventArgs e)
{
    if (!IsLoaded) return;

    if (ModeEvents.IsChecked == true) _mode = EditMode.Events;
    else if (ModeCollision.IsChecked == true) _mode = EditMode.Collision;
    else _mode = EditMode.Tiles;

    StatusText.Text = $"Mode: {_mode}";
}

private void OnOverlayChanged(object sender, RoutedEventArgs e)
{
    _showCollisionOverlay = ShowCollisionOverlay.IsChecked == true;
    _showEventsOverlay = ShowEventsOverlay.IsChecked == true;
    RequestViewportRender();
}

// -----
// Tileset loading
// -----

```

```
private void OnLoadTilesetsClick(object sender, RoutedEventArgs e)
{
    if (_currentMap is null)
    {
        StatusText.Text = "Load or create a map first.";
        return;
    }
}
```

```
Directory.CreateDirectory(PngsFolder);
```

```
var dlg = new OpenFileDialog
{
    Title = "Load Tileset PNG(s)",
    Filter = "PNG (*.png)|*.png|All files (*.*)|*.*",
    InitialDirectory = PngsFolder,
    CheckFileExists = true,
    Multiselect = true
};
```

```
if (dlg.ShowDialog(this) != true)
    return;
```

```
int added = 0;
```

```
int reenabled = 0;
```

```
foreach (var file in dlg.FileNames)
{
    if (!File.Exists(file)) continue;
```



```
    if (TryAddTilesetFromFile(file, out _, out var changed))
    {
        added++;
    }
    else
    {
        if (changed) reenabled++;
    }
}

if (added > 0 || reenabled > 0)
{
    MarkTilesetsDirty();
    RefreshTilesetPanel();
    BuildTilePalette();
    PersistMap();
    RequestViewportRender();

    StatusText.Text = $"Tilesets updated. Added: {added}, Re-enabled: {reenabled}.";
}
else
{
    RefreshTilesetPanel();
    BuildTilePalette();
    RequestViewportRender();
}
}
```

```
private bool TryAddTilesetFromFile(string absoluteFile, out string statusMessage, out
bool stateChanged)
```

```
{
```

```
    statusMessage = "";
```

```
    stateChanged = false;
```

```
    if (_currentMap is null)
```

```
    {
```

```
        statusMessage = "No map is loaded.";
```

```
        return false;
```

```
    }
```

```
    string rel;
```

```
    try
```

```
    {
```

```
        rel = IOPath.GetRelativePath(ProjectRoot, absoluteFile);
```

```
    }
```

```
    catch
```

```
    {
```

```
        statusMessage = $"Tileset path is invalid: {absoluteFile}";
```

```
        return false;
```

```
    }
```

```
    if (rel.StartsWith(".", StringComparison.Ordinal))
```

```
    {
```

```
        statusMessage = "Tileset must be inside the project folder (Vitriol.Projects).";
```

```
        return false;
```

```
}
```

```
rel = rel.Replace("\\", '/');
```

```
var relKey = NormalizeTilesetKey(rel);
```

```
var existingPaths = new HashSet<string>(
    (_currentMap.Tilesets ?? Array.Empty<TilesetRefDto>())
        .Select(t => NormalizeTilesetKey(t.ImagePath ?? "")),
    StringComparer.OrdinalIgnoreCase);
```

```
if (existingPaths.Contains(relKey))
```

```
{
```

```
    if (!IsTilesetLoaded(relKey))
```

```
    {
```

```
        _tilesetLoadedState[relKey] = true;
```

```
        stateChanged = true;
```

```
    }
```

```
    statusMessage = $"Tileset already referenced: {rel}";
```

```
    return false;
```

```
}
```

```
var baseId = IOPath.GetFileNameWithoutExtension(absoluteFile);
```

```
if (string.IsNullOrEmpty(baseId)) baseId = "tileset";
```

```
var existingIds = new HashSet<string>(
```

```
    (_currentMap.Tilesets ?? Array.Empty<TilesetRefDto>()).Select(t => t.TilesetId),
```

```
    StringComparer.OrdinalIgnoreCase);
```

```

string id = baseId;

int n = 2;

while (existingIds.Contains(id))

{
    id = $"{baseId}_{n}";
    n++;
}


var newTs = new TilesetRefDto(
    TilesetId: id,
    ImagePath: rel,
    TileWidth: 32,
    TileHeight: 32,
    Margin: 0,
    Spacing: 0
);


var list = (_currentMap.Tilesets ?? Array.Empty<TilesetRefDto>()).ToList();
list.Add(newTs);
_currentMap = _currentMap with { Tilesets = list.ToArray() };


_tilesetLoadedState[relKey] = true;


statusMessage = $"Tileset added: {id}";
stateChanged = true;
return true;
}

```

```

// -----
// Tile palette
// -----

private void BuildTilePalette()
{
    TilePalettePanel.Children.Clear();

    _selectedPaletteBorder = null;

    var map = _currentMap;
    if (map is null)
        return;

    EnsureTilesetsCompiled();

    bool any = false;

    foreach (var ts in _compiledTilesets)
    {
        if (!ts.IsLoaded || ts.Bitmap is null || ts.TileCount <= 0 || ts.CropCache is null)
            continue;

        any = true;

        for (int local = 0; local < ts.TileCount; local++)
        {
            int global = ts.StartIndexGlobal + local;

```

```

var crop = ts.CropCache.GetOrAdd(local, tileIndex =>
{
    int tx = ts.TilesAcross > 0 ? (tileIndex % ts.TilesAcross) : 0;
    int ty = ts.TilesAcross > 0 ? (tileIndex / ts.TilesAcross) : 0;

    int px = ts.Margin + tx * (ts.TileW + ts.Spacing);
    int py = ts.Margin + ty * (ts.TileH + ts.Spacing);

    if (px + ts.TileW > ts.Bitmap.PixelWidth || py + ts.TileH > ts.Bitmap.PixelHeight)
    {
        var dummy = new CroppedBitmap(ts.Bitmap, new Int32Rect(0, 0, 1, 1));
        dummy.Freeze();
        return dummy;
    }

    var cb = new CroppedBitmap(ts.Bitmap, new Int32Rect(px, py, ts.TileW,
ts.TileH));
    cb.Freeze();
    return cb;
});

var img = new Image
{
    Width = 32,
    Height = 32,
    Source = crop,
    Stretch = Stretch.Fill,
    SnapsToDevicePixels = true

```

```

};

RenderOptions.SetBitmapScalingMode(img,
BitmapScalingMode.NearestNeighbor);

var border = new Border
{
    BorderBrush = Brushes.Transparent,
    BorderThickness = new Thickness(2),
    Margin = new Thickness(4),
    Child = img,
    Tag = global
};

border.MouseLeftButtonDown += (_, _) => SelectPaletteTile(border);
TilePalettePanel.Children.Add(border);

if (global == _selectedTileIndex)
    SelectPaletteTile(border, updateStatus: false);
}
}

if (!any)
    StatusText.Text = "No tilesets are currently enabled (palette is empty).";
}

private void SelectPaletteTile(Border border, bool updateStatus = true)
{
    if (_selectedPaletteBorder != null)

```

```

        _selectedPaletteBorder.BorderBrush = Brushes.Transparent;

        _selectedPaletteBorder = border;
        _selectedPaletteBorder.BorderBrush = Brushes.Gold;

        _selectedTileIndex = (int)border.Tag;

        if (updateStatus)
            StatusText.Text = $"Selected tile: {_selectedTileIndex}";
    }

    // -----
    // Viewport rendering
    // -----

    private void EnsureViewportImageExists()
    {
        if (_viewportImage != null) return;

        _viewportImage = new Image
        {
            Stretch = Stretch.None,
            SnapsToDevicePixels = true
        };

        RenderOptions.SetBitmapScalingMode(_viewportImage,
        BitmapScalingMode.NearestNeighbor);

        MapCanvas.Children.Clear();
        MapCanvas.Children.Add(_viewportImage);
    }

```



```
// If overlays were created earlier, re-add them if they lost their parent.  
if (_dragGhostShape is not null && _dragGhostShape.Parent is null)  
    MapCanvas.Children.Add(_dragGhostShape);  
if (_dragGhostOutlineShape is not null && _dragGhostOutlineShape.Parent is null)  
    MapCanvas.Children.Add(_dragGhostOutlineShape);  
  
}
```

```
private void RequestViewportRender()  
{  
    if (_currentMap is null) return;  
    _renderDebounce.Stop();  
    _renderDebounce.Start();  
}
```

```
private void OnScrollChanged(object sender, ScrollChangedEventArgs e)  
{  
    if (_currentMap is null) return;  
    RequestViewportRender();  
}
```

```
private void RenderViewport()  
{  
    var map = _currentMap;  
    if (map is null) return;  
  
    EnsureViewportImageExists();  
}
```

```
EnsureTilesetsCompiled();
```

```
int mapPxW = checked(map.Width * map.TileSize);
```

```
int mapPxH = checked(map.Height * map.TileSize);
```

```
MapCanvas.Width = mapPxW;
```

```
MapCanvas.Height = mapPxH;
```

```
var bg = GetBackgroundLayer();
```

```
if (bg is null) return;
```

```
var col = GetCollisionLayer();
```

```
double viewLeftWorld = MapScroll.HorizontalOffset / _zoom;
```

```
double viewTopWorld = MapScroll.VerticalOffset / _zoom;
```

```
double viewWidthWorld = MapScroll.ViewportWidth / _zoom;
```

```
double viewHeightWorld = MapScroll.ViewportHeight / _zoom;
```

```
int pad = map.TileSize * 2;
```

```
int regionX = ClampToInt(viewLeftWorld) - pad;
```

```
int regionY = ClampToInt(viewTopWorld) - pad;
```

```
int regionW = ClampToInt(viewWidthWorld) + pad * 2;
```

```
int regionH = ClampToInt(viewHeightWorld) + pad * 2;
```

```
if (regionX < 0) regionX = 0;
```

```
if (regionY < 0) regionY = 0;
```

```
if (regionX + regionW > mapPxW) regionW = mapPxW - regionX;
```

```
if (regionY + regionH > mapPxH) regionH = mapPxH - regionY;
```

```
if (regionW <= 1 || regionH <= 1) return;
```

```
int startTileX = regionX / map.TileSize;
```

```
int startTileY = regionY / map.TileSize;
```

```
int endTileX = (regionX + regionW + map.TileSize - 1) / map.TileSize;
```

```
int endTileY = (regionY + regionH + map.TileSize - 1) / map.TileSize;
```

```
startTileX = Math.Max(0, startTileX);
```

```
startTileY = Math.Max(0, startTileY);
```

```
endTileX = Math.Min(map.Width, endTileX);
```

```
endTileY = Math.Min(map.Height, endTileY);
```

```
var visual = new DrawingVisual();
```

```
using (var dc = visual.RenderOpen())
```

```
{
```

```
    dc.DrawRectangle(Brushes.White, null, new Rect(0, 0, regionW, regionH));
```

```
    for (int ty = startTileY; ty < endTileY; ty++)
```

```
        for (int tx = startTileX; tx < endTileX; tx++)
```

```
        {
```

```
            int idx = ty * map.Width + tx;
```

```
            int t = bg.Tiles[idx];
```

```
            if (t < 0) continue;
```

```
            var ts = FindRuntimeForGlobalTile(t);
```

```
            if (ts is null) continue;
```

```

if (!ts.IsLoaded || ts.Bitmap is null || ts.CropCache is null || ts.TileCount <= 0)
    continue;

int local = GetLocalIndex(ts, t);
if (local < 0 || local >= ts.TileCount) continue;

var crop = ts.CropCache.GetOrAdd(local, tileIndex =>
{
    int cx = ts.TilesAcross > 0 ? (tileIndex % ts.TilesAcross) : 0;
    int cy = ts.TilesAcross > 0 ? (tileIndex / ts.TilesAcross) : 0;

    int px = ts.Margin + cx * (ts.TileW + ts.Spacing);
    int py = ts.Margin + cy * (ts.TileH + ts.Spacing);

    if (px + ts.TileW > ts.Bitmap.PixelWidth || py + ts.TileH > ts.Bitmap.PixelHeight)
    {
        var dummy = new CroppedBitmap(ts.Bitmap, new Int32Rect(0, 0, 1, 1));
        dummy.Freeze();
        return dummy;
    }

    var cb = new CroppedBitmap(ts.Bitmap, new Int32Rect(px, py, ts.TileW,
ts.TileH));
    cb.Freeze();
    return cb;
});

double drawX = tx * map.TileSize - regionX;

```

```

double drawY = ty * map.TileSize - regionY;

dc.DrawImage(crop, new Rect(drawX, drawY, map.TileSize, map.TileSize));
}

if (_showCollisionOverlay && col is not null && col.Tiles.Length == map.Width *
map.Height)
{
    var brush = new SolidColorBrush(Color.FromArgb(110, 255, 0, 0));
    brush.Freeze();

    for (int ty = startTileY; ty < endTileY; ty++)
    for (int tx = startTileX; tx < endTileX; tx++)
    {
        int idx = ty * map.Width + tx;

        if (idx < 0 || idx >= col.Tiles.Length) continue;

        if (col.Tiles[idx] != 0)
        {
            double drawX = tx * map.TileSize - regionX;
            double drawY = ty * map.TileSize - regionY;

            dc.DrawRectangle(brush, null, new Rect(drawX, drawY, map.TileSize,
map.TileSize));
        }
    }
}

if (_showEventsOverlay && map.Events is not null && map.Events.Count > 0)
{

```

```

var triggerFill = new SolidColorBrush(Color.FromArgb(220, 255, 215, 0));
triggerFill.Freeze();

var npcFill = new SolidColorBrush(Color.FromArgb(220, 80, 170, 255));
npcFill.Freeze();

var warpFill = new SolidColorBrush(Color.FromArgb(220, 190, 90, 255));
warpFill.Freeze();

var outlinePen = new Pen(new SolidColorBrush(Color.FromArgb(255, 10, 10,
10)), 1);
outlinePen.Freeze();

var selPen = new Pen(Brushes.White, 2);
selPen.Freeze();

foreach (var ev in map.Events)
{
    if (ev.X < startTileX || ev.X >= endTileX || ev.Y < startTileY || ev.Y >= endTileY)
        continue;

    double tileLeft = ev.X * map.TileSize - regionX;
    double tileTop = ev.Y * map.TileSize - regionY;

    bool isSelected = !string.IsNullOrEmpty(_selectedEventId) && ev.EventId
== _selectedEventId;

    var type = (ev.Type ?? "").Trim();

    if (string.Equals(type, "NPC", StringComparison.OrdinalIgnoreCase))

```

```

{
    var center = new Point(tileLeft + map.TileSize / 2.0, tileTop + map.TileSize /
2.0);

    double r = map.TileSize * 0.22;

    dc.DrawEllipse(npcFill, outlinePen, center, r, r);

    if (isSelected)
        dc.DrawRectangle(null, selPen, new Rect(tileLeft + 1, tileTop + 1,
map.TileSize - 2, map.TileSize - 2));
}
else if (string.Equals(type, "Warp", StringComparison.OrdinalIgnoreCase))
{
    var cx = tileLeft + map.TileSize / 2.0;
    var cy = tileTop + map.TileSize / 2.0;
    double r = map.TileSize * 0.24;

    var geo = new StreamGeometry();
    using (var g = geo.Open())
    {
        g.BeginFigure(new Point(cx, cy - r), isFilled: true, isClosed: true);
        g.LineTo(new Point(cx + r, cy), isStroked: true, isSmoothJoin: false);
        g.LineTo(new Point(cx, cy + r), isStroked: true, isSmoothJoin: false);
        g.LineTo(new Point(cx - r, cy), isStroked: true, isSmoothJoin: false);
    }
    geo.Freeze();

    dc.DrawGeometry(warpFill, outlinePen, geo);
}

```

```

        if (isSelected)

            dc.DrawRectangle(null, selPen, new Rect(tileLeft + 1, tileTop + 1,
map.TileSize - 2, map.TileSize - 2));

        }

        else

        {

            var rr = new Rect(

                tileLeft + map.TileSize * 0.28,

                tileTop + map.TileSize * 0.28,

                map.TileSize * 0.44,

                map.TileSize * 0.44);

            dc.DrawRectangle(triggerFill, outlinePen, rr);

            if (isSelected)

                dc.DrawRectangle(null, selPen, new Rect(tileLeft + 1, tileTop + 1,
map.TileSize - 2, map.TileSize - 2));

        }

    }

}

if (_playtestActive)

{

    if (_playerX >= startTileX && _playerX < endTileX && _playerY >= startTileY &&
_playerY < endTileY)

    {

        double px = _playerX * map.TileSize - regionX;

        double py = _playerY * map.TileSize - regionY;

```



```

var fill = new SolidColorBrush(Color.FromArgb(220, 90, 180, 255));
fill.Freeze();

var pen = new Pen(Brushes.Black, 1);
pen.Freeze();

dc.DrawEllipse(fill, pen,
    new Point(px + map.TileSize / 2.0, py + map.TileSize / 2.0),
    map.TileSize * 0.28,
    map.TileSize * 0.28);
}
}

var gridBrush = new SolidColorBrush(Color.FromArgb(90, 0, 0, 0));
gridBrush.Freeze();

var gridPen = new Pen(gridBrush, 1);
gridPen.Freeze();

RenderOptions.SetEdgeMode(visual, EdgeMode.Aliased);

var guidelines = new GuidelineSet();

for (int tx = startTileX; tx <= endTileX; tx++)
{
    double x = (tx * map.TileSize - regionX);
    guidelines.GuidelinesX.Add(x);
}

```

```

for (int ty = startTileY; ty <= endTileY; ty++)
{
    double y = (ty * map.TileSize - regionY);
    guidelines.GuidelinesY.Add(y);
}

dc.PushGuidelineSet(guidelines);

for (int tx = startTileX; tx <= endTileX; tx++)
{
    double x = (tx * map.TileSize - regionX);
    dc.DrawLine(gridPen, new Point(x, 0), new Point(x, regionH));
}

for (int ty = startTileY; ty <= endTileY; ty++)
{
    double y = (ty * map.TileSize - regionY);
    dc.DrawLine(gridPen, new Point(0, y), new Point(regionW, y));
}

dc.Pop();
}

var rtb = new RenderTargetBitmap(regionW, regionH, 96, 96, PixelFormats.Pbgra32);
rtb.Render(visual);
rtb.Freeze();

_viewportImage!.Source = rtb;

```

```
_viewportImage.Width = regionW;  
_viewportImage.Height = regionH;
```

```
Canvas.SetLeft(_viewportImage, regionX);  
Canvas.SetTop(_viewportImage, regionY);  
}
```

```
private static int ClampToInt(double v)  
{  
    if (v <= int.MinValue) return int.MinValue;  
    if (v >= int.MaxValue) return int.MaxValue;  
    return (int)Math.Floor(v);  
}
```

```
private static BitmapSource LoadBitmap(string path)  
{  
    var bmp = new BitmapImage();  
    bmp.BeginInit();  
    bmp.CacheOption = BitmapCacheOption.OnLoad;  
    bmp.UriSource = new Uri(path, UriKind.Absolute);  
    bmp.EndInit();  
    bmp.Freeze();  
    return bmp;  
}
```

```
private void OnSaveMapClick(object sender, RoutedEventArgs e)  
{  
    if (_currentMap is null)
```

```

{
    StatusText.Text = "Nothing to save (no map loaded).";
    return;
}

Directory.CreateDirectory(MapsFolder);

if (string.IsNullOrEmpty(_currentMapPath))
{
    var dlg = new SaveFileDialog
    {
        Title = "Save Vitriol Map",
        Filter = "Vitriol Map (*.vitmap.json)|*.vitmap.json|JSON (*.json)|*.json|All files (*.*)|*.*",
        InitialDirectory = MapsFolder,
        FileName = (_currentMap.MapId ?? "new_map") + MapFormat.DefaultExtension,
        OverwritePrompt = true
    };

    if (dlg.ShowDialog(this) != true)
        return;

    _currentMapPath = dlg.FileName;
}

try
{
    MapSerializer.SaveToFile(_currentMapPath, _currentMap);
}

```

```

        if (!IsSampleMapLoaded())
            _manualSavePerformed = true;

        StatusText.Text = $"Saved: {_currentMapPath}";
    }
    catch (Exception ex)
    {
        StatusText.Text = $"Save failed: {ex.Message}";
    }
}

// -----
// Mouse + painting + event clicks
// -----

private void OnMapCanvasMouseLeftButtonDown(object sender,
MouseButtonEventArgs e)
{
    if (_currentMap is null) return;

    if (_playtestActive)
    {
        StatusText.Text = "Playtest is active (F5 to stop). Editing is disabled.";
        return;
    }

    var p = e.GetPosition(MapCanvas);
    SetCursorFromPoint(p);

```

```
if (_mode == EditMode.Events)
{
    HandleEventLeftClick(p);
    return;
}
```

```
StartPainting();
PaintAtMouse(p, isRightButton: false);
}
```

```
private void OnMapCanvasMouseRightButtonDown(object sender,
MouseButtonEventArgs e)
{
    if (_currentMap is null) return;

    if (_playtestActive)
    {
        StatusText.Text = "Playtest is active (F5 to stop). Editing is disabled.";
        return;
    }
```

```
var p = e.GetPosition(MapCanvas);
SetCursorFromPoint(p);
```

```
if (_mode == EditMode.Events)
{
    _selectedEventId = null;
```

```
        _selectedCommandIndex = -1;
        UpdateEventPanel(null);
        RefreshCommandList(null);
        UpdateCommandEditor(null);
        RequestViewportRender();
        return;
    }
```

```
        StartPainting();
        PaintAtMouse(p, isRightButton: true);
    }
```

```
private void OnMapCanvasMouseLeftButtonUp(object sender,
MouseButtonEventArgs e)
```

```
{
    StopPainting();
    StopEventDrag(commit: true);
}
```

```
private void StartPainting()
```

```
{
    _isPainting = true;
    _lastPaintIdx = -1;
    MapCanvas.CaptureMouse();
}
```

```
private void StopPainting()
```

```
{
```

```

_isPainting = false;

_lastPaintIdx = -1;

if (Mouse.Captured == MapCanvas)
    MapCanvas.ReleaseMouseCapture();
}

private void OnMapCanvasMouseMove(object sender, MouseEventArgs e)
{
    var map = _currentMap;
    if (map is null) return;

    var p = e.GetPosition(MapCanvas);

    int tileX = (int)Math.Floor(p.X / map.TileSize);
    int tileY = (int)Math.Floor(p.Y / map.TileSize);
    _hoverTileX = tileX;
    _hoverTileY = tileY;

    if (tileX < 0 || tileY < 0 || tileX >= map.Width || tileY >= map.Height)
    {
        MouseText.Text = "";
        _hoverTileX = -1;
        _hoverTileY = -1;

        // If dragging, clamp so ghost doesn't disappear
        if (_draggingEvent)
        {

```



```

        _dragMouseWorldX = p.X;
        _dragMouseWorldY = p.Y;
        UpdateDragOverlays();
        UpdateDragViewportPoint(e.GetPosition(MapScroll));
    }

    return;
}

if (_mode == EditMode.Events && _draggingEvent)
{
    _dragMouseWorldX = p.X;
    _dragMouseWorldY = p.Y;

    UpdateDragOverlays();
    UpdateDragViewportPoint(e.GetPosition(MapScroll));
}

int idx = tileY * map.Width + tileX;
int colVal = GetCollisionLayer()?.Tiles.ElementAtOrDefault(idx) ?? 0;

var hoverEv = GetEventAt(tileX, tileY);
var hoverText = (hoverEv is null) ? "" : " | " + DescribeEventForHover(hoverEv);

MouseText.Text =
    $"Tile: ({tileX},{tileY}) Mode:{_mode} Col:{colVal}" +
    (_playtestActive ? $" Player:({_playerX},{_playerY})" : "") +
    hoverText;

```

```

if (_isPainting && _mode != EditMode.Events && !_playtestActive)
{
    bool right = e.RightButton == MouseButtonState.Pressed;
    bool left = e.LeftButton == MouseButtonState.Pressed;
    if (left || right)
        PaintAtMouse(p, isRightButton: right && !left);
}
}

```

```

private void SetCursorFromPoint(Point p)
{
    var map = _currentMap;
    if (map is null) return;

    int tileX = (int)Math.Floor(p.X / map.TileSize);
    int tileY = (int)Math.Floor(p.Y / map.TileSize);

    if (tileX < 0 || tileY < 0 || tileX >= map.Width || tileY >= map.Height)
        return;

    _cursorTileX = tileX;
    _cursorTileY = tileY;
}

```

```

private void PaintAtMouse(Point p, bool isRightButton)
{
    var map = _currentMap;

```

```

if (map is null) return;

int tileX = (int)Math.Floor(p.X / map.TileSize);
int tileY = (int)Math.Floor(p.Y / map.TileSize);
if (tileX < 0 || tileY < 0 || tileX >= map.Width || tileY >= map.Height)
    return;

int idx = tileY * map.Width + tileX;

if (idx == _lastPaintIdx) return;
_lastPaintIdx = idx;

if (_mode == EditMode.Tiles)
{
    var bg = GetBackgroundLayer();
    if (bg is null) return;

    int oldValue = bg.Tiles[idx];
    int newValue = _selectedTileIndex;
    if (oldValue == newValue) return;

    ExecuteCommand(new PaintIntCellCommand(this, LayerKind.Background, idx,
oldValue, newValue));
}

else if (_mode == EditMode.Collision)
{
    var col = GetCollisionLayer();
    if (col is null) return;

```

```

        int oldValue = col.Tiles[idx];

        int newValue = isRightButton ? 0 : 1;

        if (oldValue == newValue) return;

        ExecuteCommand(new PaintIntCellCommand(this, col.Kind, idx, oldValue,
newValue));
    }
}

private void OnCreateNewMapClick(object sender, RoutedEventArgs e)
{
    if (!TryReadMapSize(out int w, out int h))
        return;

    Directory.CreateDirectory(MapsFolder);

    var dlg = new SaveFileDialog
    {
        Title = "Create New Map",
        Filter = "Vitriol Map (*.vitmap.json)|*.vitmap.json|JSON (*.json)|*.json|All files (*.*)|*.*",
        InitialDirectory = MapsFolder,
        FileName = "new_map" + MapFormat.DefaultExtension,
        OverwritePrompt = false
    };

    if (dlg.ShowDialog(this) != true)

```

```
    return;

    var chosenPath = dlg.FileName;

    var baseName = IOPath.GetFileName(chosenPath);

    if (baseName.EndsWith(".json", StringComparison.OrdinalIgnoreCase))
        baseName = baseName[..^5];

    if (baseName.EndsWith(".vitmap", StringComparison.OrdinalIgnoreCase))
        baseName = baseName[..^7];

    baseName = baseName.Trim();
    if (baseName.Length == 0) baseName = "new_map";

    var mapId = MakeSafeMapId(baseName);
    var mapName = baseName;

    var map = CreateNewBlankMap(mapId, mapName, w, h);

    var errors = MapValidator.Validate(map);
    if (errors.Count > 0)
    {
        StatusText.Text = "Map validation failed: " +
            string.Join("; ", errors.Select(er => $"{er.Field}: {er.Message}"));

        return;
    }
```

```
_currentMap = map;
_currentMapPath = chosenPath;
_manualSavePerformed = false;

if (_currentMap.Events is null)
    _currentMap = _currentMap with { Events = Array.Empty<MapEventDto>() };

_selectedEventId = null;
_selectedCommandIndex = -1;
UpdateEventPanel(null);
RefreshCommandList(null);
UpdateCommandEditor(null);

EnsureCollisionLayer();

_undo.Clear();
_redo.Clear();

var tilesets = _currentMap.Tilesets ?? Array.Empty<TilesetRefDto>();
foreach (var ts in tilesets)
{
    var key = NormalizeTilesetKey(ts.ImagePath ?? "");
    if (key.Length == 0) continue;
    _tilesetLoadedState.TryAdd(key, true);
}

MarkTilesetsDirty();
```

```
EnsureViewportImageExists();
```

```
RefreshTilesetPanel();
```

```
BuildTilePalette();
```

```
MapScroll.ScrollToHorizontalOffset(0);
```

```
MapScroll.ScrollToVerticalOffset(0);
```

```
RefreshAvailableMaps(preserveCurrentSelection: false);
```

```
StatusText.Text = $"Created new map (not yet saved): {chosenPath} — press Save to  
enable autosave.";
```

```
RequestViewportRender();
```

```
}
```

```
// XAML handler: do not remove / rename
```

```
private void OnMapCanvasMouseLeave(object sender, MouseEventArgs e)
```

```
{
```

```
    MouseText.Text = "";
```

```
    _hoverTileX = -1;
```

```
    _hoverTileY = -1;
```

```
    StopPainting();
```

```
    StopEventDrag(commit: false);
```

```
}
```

```
// -----
```

```
// Playtest execution
```

```
// -----
```

```

private void TogglePlaytest()
{
    if (_currentMap is null)
    {
        StatusText.Text = "Load or create a map first.";
        return;
    }

    _playtestActive = !_playtestActive;

    if (_playtestActive)
    {
        _playtestFlags.Clear();
        _playerX = Math.Clamp(_playerX, 0, _currentMap.Width - 1);
        _playerY = Math.Clamp(_playerY, 0, _currentMap.Height - 1);
        StatusText.Text = "Playtest ON — F5 toggles, WASD/Arrows move, E/Enter interact,
Esc exit.";
    }
    else
    {
        StatusText.Text = "Playtest OFF.";
    }

    RequestViewportRender();
}

private void ExitPlaytest()
{

```



```
if (!_playtestActive) return;
_playtestActive = false;
StatusText.Text = "Playtest OFF.";
RequestViewportRender();
}
```

```
private void TryMovePlayer(int dx, int dy)
```

```
{
    var map = _currentMap;
    if (map is null) return;
    if (!_playtestActive) return;
```

```
    int nx = _playerX + dx;
```

```
    int ny = _playerY + dy;
```

```
    if (nx < 0 || ny < 0 || nx >= map.Width || ny >= map.Height)
```

```
{
    StatusText.Text = "Blocked: map boundary.";
    return;
}
```

```
var col = GetCollisionLayer();
```

```
if (col is not null && col.Tiles.Length == map.Width * map.Height)
```

```
{
    int idx = ny * map.Width + nx;
    if (idx >= 0 && idx < col.Tiles.Length && col.Tiles[idx] != 0)
    {
        StatusText.Text = "Blocked: collision tile.";
    }
}
```

```
        return;  
    }  
}
```

```
_playerX = nx;  
_playerY = ny;
```

```
RequestViewportRender();
```

```
var ev = GetEventAt(_playerX, _playerY);  
if (ev is null) return;
```

```
if (string.Equals(ev.Type, "Warp", StringComparison.OrdinalIgnoreCase))  
{  
    DoWarp(ev);  
    return;  
}
```

```
if (string.Equals(ev.Type, "Trigger", StringComparison.OrdinalIgnoreCase))  
    ExecuteEventScript(ev);  
}
```

```
private void InteractAtPlayer()  
{  
    var ev = GetEventAt(_playerX, _playerY);  
    if (ev is null)  
    {  
        StatusText.Text = "Interact: no event present.";
```

```
    return;  
}
```

```
if (string.Equals(ev.Type, "NPC", StringComparison.OrdinalIgnoreCase) ||  
    string.Equals(ev.Type, "Trigger", StringComparison.OrdinalIgnoreCase))  
{  
    ExecuteEventScript(ev);  
    return;  
}
```

```
StatusText.Text = $"Interact: {ev.Name} ({ev.Type})";  
}
```

```
private static string DescribeEventForHover(MapEventDto ev)  
{  
    var type = (ev.Type ?? "Trigger").Trim();  
    var name = string.IsNullOrEmpty(ev.Name) ? "Event" : ev.Name;  
  
    if (type.Equals("Warp", StringComparison.OrdinalIgnoreCase))  
    {  
        var p = ev.Parameters ?? new Dictionary<string, string>();  
        p.TryGetValue("destMap", out var dm);  
        p.TryGetValue("destX", out var dx);  
        p.TryGetValue("destY", out var dy);  
  
        dm = string.IsNullOrEmpty(dm) ? "?" : dm;  
        dx = string.IsNullOrEmpty(dx) ? "?" : dx;  
        dy = string.IsNullOrEmpty(dy) ? "?" : dy;
```

```
    return $"Event: {name} (Warp → {dm} @ {dx},{dy})";  
}
```

```
    return $"Event: {name} ({type})";  
}
```

```
private MapEventDto? GetEventAt(int x, int y)  
{  
    var map = _currentMap;  
    if (map is null) return null;  
    if (map.Events is null || map.Events.Count == 0) return null;  
    return map.Events.FirstOrDefault(e => e.X == x && e.Y == y);  
}
```

```
private void ExecuteEventScript(MapEventDto ev)  
{  
    var cmds = ev.Commands ?? Array.Empty<EventCommandDto>();  
    if (cmds.Count == 0)  
    {  
        StatusText.Text = $"Event '{ev.Name}' has no script commands.";  
        return;  
    }  
}
```

```
foreach (var cmd in cmds)  
{  
    switch ((cmd.Command ?? "").Trim())  
    {
```

```
case "ShowText":  
  
{  
    cmd.Args.TryGetValue("text", out var text);  
    text ??= "";  
    if (text.Length == 0) text = "(empty text)";  
    MessageBox.Show(this, text, ev.Name);  
    break;  
}
```

```
case "SetFlag":  
  
{  
    cmd.Args.TryGetValue("flag", out var flag);  
    cmd.Args.TryGetValue("value", out var value);  
    flag = (flag ?? "").Trim();  
    value ??= "";  
  
    if (flag.Length == 0)  
    {  
        StatusText.Text = "SetFlag failed: flag is empty.";  
        break;  
    }  
  
    _playtestFlags[flag] = value;  
    StatusText.Text = $"Flag set: {flag} = '{value}'";  
    break;  
}
```

```
default:
```

```

        StatusText.Text = $"Unknown command: {cmd.Command}";
        break;
    }
}
}

```

```

private void DoWarp(MapEventDto ev)
{
    var p = ev.Parameters ?? new Dictionary<string, string>();

    p.TryGetValue("destMap", out var destMap);
    p.TryGetValue("destX", out var destXStr);
    p.TryGetValue("destY", out var destYStr);

    destMap = (destMap ?? "").Trim();
    if (string.IsNullOrEmpty(destMap))
    {
        StatusText.Text = "Warp failed: destMap missing.";
        return;
    }

    int.TryParse(destXStr, out var destX);
    int.TryParse(destYStr, out var destY);

    var destPath = IOPath.Combine(MapsFolder, destMap +
MapFormat.DefaultExtension);

    if (!File.Exists(destPath))

```

```

{
    StatusText.Text = $"Warp failed: map not found '{destMap}'";
    return;
}

LoadMapFromPath(destPath, $"Warped to {destMap}");

if (_currentMap is null) return;

_playerX = Math.Clamp(destX, 0, _currentMap.Width - 1);
_playerY = Math.Clamp(destY, 0, _currentMap.Height - 1);

StatusText.Text = $"Warped: {ev.Name} -> {destMap} ({_playerX},{_playerY})";
RequestViewportRender();
}

// -----
// Events editor + dragging
// -----

private void HandleEventLeftClick(Point p)
{
    var map = _currentMap;
    if (map is null) return;

    int tileX = (int)Math.Floor(p.X / map.TileSize);
    int tileY = (int)Math.Floor(p.Y / map.TileSize);
    if (tileX < 0 || tileY < 0 || tileX >= map.Width || tileY >= map.Height)
        return;

```

```
_cursorTileX = tileX;
```

```
_cursorTileY = tileY;
```

```
var existing = map.Events.FirstOrDefault(ev => ev.X == tileX && ev.Y == tileY);
```

```
if (existing is not null)
```

```
{
```

```
    _selectedEventId = existing.EventId;
```

```
    _selectedCommandIndex = -1;
```

```
    UpdateEventPanel(existing);
```

```
    RefreshCommandList(existing);
```

```
    UpdateCommandEditor(null);
```

```
    BeginEventDrag(existing);
```

```
    RequestViewportRender();
```

```
    return;
```

```
}
```

```
var newEv = new MapEventDto(
```

```
    EventId: Guid.NewGuid().ToString("N"),
```

```
    Name: "New Event",
```

```
    Type: "Trigger",
```

```
    X: tileX,
```

```
    Y: tileY,
```

```
    Parameters: new Dictionary<string, string>(),
```

```
    Commands: Array.Empty<EventCommandDto>()
```

```
);
```



```

ExecuteCommand(new AddEventCommand(this, newEv));

_selectedEventId = newEv.EventId;

_selectedCommandIndex = -1;


UpdateEventPanel(newEv);

RefreshCommandList(newEv);

UpdateCommandEditor(null);


RequestViewportRender();
}


private void EnsureDragOverlays()
{
if (_dragGhostShape is not null && _dragGhostOutlineShape is not null) return;


// Filled ghost (shape changes by event type)
_dragGhostShape = new System.Windows.Shapes.Rectangle
{
    Width = 18,
    Height = 18,
    RadiusX = 4,
    RadiusY = 4,
    Fill = new SolidColorBrush(Color.FromArgb(170, 255, 215, 0)), // Trigger-ish default
    StrokeThickness = 0,
    IsHitTestVisible = false,
    Visibility = Visibility.Collapsed
};

```

```
// Outline ghost (shape changes too)
_dragGhostOutlineShape = new System.Windows.Shapes.Rectangle
{
    Width = 22,
    Height = 22,
    RadiusX = 5,
    RadiusY = 5,
    Stroke = new SolidColorBrush(Color.FromArgb(220, 0, 0, 0)),
    StrokeThickness = 2,
    Fill = Brushes.Transparent,
    IsHitTestVisible = false,
    Visibility = Visibility.Collapsed
};

MapCanvas.Children.Add(_dragGhostShape);
MapCanvas.Children.Add(_dragGhostOutlineShape);
}
```

```
private void ShowDragOverlays()
{
    EnsureDragOverlays();
    _dragGhostShape!.Visibility = Visibility.Visible;
    _dragGhostOutlineShape!.Visibility = Visibility.Visible;
}
```

```
private void HideDragOverlays()
{
    if (_dragGhostShape is not null) _dragGhostShape.Visibility = Visibility.Collapsed;
    if (_dragGhostOutlineShape is not null) _dragGhostOutlineShape.Visibility =
Visibility.Collapsed;
}
```

```
private void EnsureSwapHintOverlays()
{
    if (_swapHintFrom is not null && _swapHintTo is not null) return;
```

```
_swapHintFrom = new System.Windows.Shapes.Rectangle
```

```
{
    Stroke = Brushes.Gold,
    StrokeThickness = 3,
    Fill = Brushes.Transparent,
    IsHitTestVisible = false,
    Visibility = Visibility.Collapsed
};
```

```
_swapHintTo = new System.Windows.Shapes.Rectangle
```

```
{
    Stroke = Brushes.White,
    StrokeThickness = 3,
    Fill = Brushes.Transparent,
    IsHitTestVisible = false,
```

```

        Visibility = Visibility.Collapsed
    };

    EnsureViewportImageExists();
    MapCanvas.Children.Add(_swapHintFrom);
    MapCanvas.Children.Add(_swapHintTo);

    _swapHintTimer.Interval = TimeSpan.FromMilliseconds(140);
    _swapHintTimer.Tick += (_, _) =>
    {
        _swapHintTimer.Stop();
        if (_swapHintFrom is not null) _swapHintFrom.Visibility = Visibility.Collapsed;
        if (_swapHintTo is not null) _swapHintTo.Visibility = Visibility.Collapsed;
    };
}

private void ShowSwapHint(int fromX, int fromY, int toX, int toY)
{
    var map = _currentMap;
    if (map is null) return;

    EnsureSwapHintOverlays();

    double s = map.TileSize;

    _swapHintFrom!.Width = s;
    _swapHintFrom.Height = s;

```

```

Canvas.SetLeft(_swapHintFrom, fromX * s);
Canvas.SetTop(_swapHintFrom, fromY * s);
_swapHintFrom.Visibility = Visibility.Visible;

_swapHintTo!.Width = s;
_swapHintTo.Height = s;
Canvas.SetLeft(_swapHintTo, toX * s);
Canvas.SetTop(_swapHintTo, toY * s);
_swapHintTo.Visibility = Visibility.Visible;

_swapHintTimer.Stop();
_swapHintTimer.Start();
}

```

```

private void ConfigureDragGhostShapeForEvent(MapEventDto ev)
{
    var type = (ev.Type ?? "Trigger").Trim();

```

Brush fill =

```

    string.Equals(type, "NPC", StringComparison.OrdinalIgnoreCase)
        ? new SolidColorBrush(Color.FromArgb(220, 80, 170, 255))
        : string.Equals(type, "Warp", StringComparison.OrdinalIgnoreCase)
            ? new SolidColorBrush(Color.FromArgb(220, 190, 90, 255))
            : new SolidColorBrush(Color.FromArgb(220, 255, 215, 0));

```

// Swap shape classes (Rectangle/Ellipse/Polygon) so the ghost matches type

```

if (string.Equals(type, "NPC", StringComparison.OrdinalIgnoreCase))
{
    EnsureOverlays<System.Windows.Shapes.Ellipse>(ref _dragGhostShape);
    EnsureOverlays<System.Windows.Shapes.Ellipse>(ref _dragGhostOutlineShape);

    _dragGhostShape!.Fill = fill;
    _dragGhostOutlineShape!.Fill = Brushes.Transparent;
    _dragGhostOutlineShape!.Stroke = new SolidColorBrush(Color.FromArgb(220, 0, 0,
0));
    _dragGhostOutlineShape!.StrokeThickness = 2;
}
else if (string.Equals(type, "Warp", StringComparison.OrdinalIgnoreCase))
{
    EnsureOverlays<System.Windows.Shapes.Polygon>(ref _dragGhostShape);
    EnsureOverlays<System.Windows.Shapes.Polygon>(ref _dragGhostOutlineShape);

    ((System.Windows.Shapes.Polygon)_dragGhostShape!).Fill = fill;
    ((System.Windows.Shapes.Polygon)_dragGhostShape!).StrokeThickness = 0;

    var o = (System.Windows.Shapes.Polygon)_dragGhostOutlineShape!;
    o.Fill = Brushes.Transparent;
    o.Stroke = new SolidColorBrush(Color.FromArgb(220, 0, 0, 0));
    o.StrokeThickness = 2;
}
else
{
    EnsureOverlays<System.Windows.Shapes.Rectangle>(ref _dragGhostShape);
    EnsureOverlays<System.Windows.Shapes.Rectangle>(ref
_dragGhostOutlineShape);

```

```

var r = (System.Windows.Shapes.Rectangle)_dragGhostShape!;

r.Fill = fill;

r.StrokeThickness = 0;


var ro = (System.Windows.Shapes.Rectangle)_dragGhostOutlineShape!;

ro.Fill = Brushes.Transparent;

ro.Stroke = new SolidColorBrush(Color.FromArgb(220, 0, 0, 0));

ro.StrokeThickness = 2;
}
}

private void EnsureOverlays<T>(ref System.Windows.Shapes.Shape? shape) where T :
System.Windows.Shapes.Shape, new()
{
    if (shape is T) return;

    // preserve current z-order by replacing in-place

    int idx = -1;

    if (shape is not null) idx = MapCanvas.Children.IndexOf(shape);

    var newShape = new T
    {
        IsHitTestVisible = false,
        Visibility = shape?.Visibility ?? Visibility.Collapsed
    };

    // carry over basic styling defaults

```

```
if (newShape is System.Windows.Shapes.Rectangle rr)
{
    rr.RadiusX = 4;
    rr.RadiusY = 4;
}

if (idx >= 0)
{
    MapCanvas.Children.RemoveAt(idx);
    MapCanvas.Children.Insert(idx, newShape);
}
else
{
    MapCanvas.Children.Add(newShape);
}

shape = newShape;
}
```

```
private void BeginEventDrag(MapEventDto ev)
{
    var map = _currentMap;
    if (map is null) return;

    _draggingEvent = true;
    _dragEventId = ev.EventId;
```



```

var mouse = Mouse.GetPosition(MapCanvas);

_dragMouseWorldX = mouse.X;
_dragMouseWorldY = mouse.Y;

_dragPreviewX = ev.X;
_dragPreviewY = ev.Y;

MapCanvas.CaptureMouse();
ShowDragOverlays();
ConfigureDragGhostShapeForEvent(ev);
UpdateDragOverlays();

StatusText.Text = $"Dragging event '{ev.Name}' — release to drop.";
}

private void UpdateDragOverlays()
{
var map = _currentMap;
if (map is null) return;
if (!_draggingEvent) return;
if (_dragGhostShape is null || _dragGhostOutlineShape is null) return;

var ev = GetSelectedEvent();
if (ev is not null)
    ConfigureDragGhostShapeForEvent(ev);

// Snap preview to tile but clamp to map bounds so it never disappears
int tx = (int)Math.Floor(_dragMouseWorldX / map.TileSize);

```

```
int ty = (int)Math.Floor(_dragMouseWorldY / map.TileSize);

tx = Math.Clamp(tx, 0, map.Width - 1);
ty = Math.Clamp(ty, 0, map.Height - 1);

_dragPreviewX = tx;
_dragPreviewY = ty;

// Ghost follows mouse, but if mouse is outside map, clamp ghost to tile center
double cx = _dragMouseWorldX;
double cy = _dragMouseWorldY;

if (_dragMouseWorldX < 0 || _dragMouseWorldY < 0 ||
    _dragMouseWorldX >= map.Width * map.TileSize ||
    _dragMouseWorldY >= map.Height * map.TileSize)
{
    cx = tx * map.TileSize + map.TileSize / 2.0;
    cy = ty * map.TileSize + map.TileSize / 2.0;
}

double inner = map.TileSize * 0.40;
double outer = map.TileSize * 0.48;

_dragGhostShape.Width = inner;
_dragGhostShape.Height = inner;

_dragGhostOutlineShape.Width = outer;
_dragGhostOutlineShape.Height = outer;
```

```

// Warp uses diamond points
if (_dragGhostShape is System.Windows.Shapes.Polygon p1)
{
    p1.Points = new PointCollection
    {
        new Point(inner/2, 0),
        new Point(inner, inner/2),
        new Point(inner/2, inner),
        new Point(0, inner/2)
    };
}

if (_dragGhostOutlineShape is System.Windows.Shapes.Polygon p2)
{
    p2.Points = new PointCollection
    {
        new Point(outer/2, 0),
        new Point(outer, outer/2),
        new Point(outer/2, outer),
        new Point(0, outer/2)
    };
}

// Trigger uses rounded rect
if (_dragGhostShape is System.Windows.Shapes.Rectangle r1)
{
    r1.RadiusX = Math.Max(2, inner * 0.18);
    r1.RadiusY = Math.Max(2, inner * 0.18);
}

```

```

    }

    if (_dragGhostOutlineShape is System.Windows.Shapes.Rectangle r2)
    {
        r2.RadiusX = Math.Max(2, outer * 0.18);
        r2.RadiusY = Math.Max(2, outer * 0.18);
    }

    Canvas.SetLeft(_dragGhostShape, cx - inner / 2.0);
    Canvas.SetTop(_dragGhostShape, cy - inner / 2.0);

    Canvas.SetLeft(_dragGhostOutlineShape, cx - outer / 2.0);
    Canvas.SetTop(_dragGhostOutlineShape, cy - outer / 2.0);
}

```

```

private void StopEventDrag(bool commit)
{
    if (!_draggingEvent)
        return;

    try
    {
        if (!commit)
            return;

        var ev = GetSelectedEvent();
        if (ev is null || _dragEventId is null || ev.EventId != _dragEventId)
            return;
    }
}

```

```

var map = _currentMap;

if (map is null) return;

int nx = _dragPreviewX;
int ny = _dragPreviewY;

nx = Math.Clamp(nx, 0, map.Width - 1);
ny = Math.Clamp(ny, 0, map.Height - 1);

if (nx == ev.X && ny == ev.Y)
    return;

// Prevent stacking events by swapping if needed
var other = GetEventAt(nx, ny);
if (other is not null && other.EventId != ev.EventId)
{
    var updatedA = ev with { X = nx, Y = ny };
    var updatedB = other with { X = ev.X, Y = ev.Y };

    ExecuteCommand(new SwapEventsCommand(this, ev, updatedA, other,
updatedB));

    _selectedEventId = updatedA.EventId;
    UpdateEventPanel(updatedA);
    RefreshCommandList(updatedA);
    UpdateCommandEditor(null);

```

```

        StatusText.Text = $"Events swapped.";
        return;
    }

    var updated = ev with { X = nx, Y = ny };
    ExecuteCommand(new ReplaceEventCommand(this, ev, updated));
    _selectedEventId = updated.EventId;

    UpdateEventPanel(updated);
    RefreshCommandList(updated);
    UpdateCommandEditor(null);

    StatusText.Text = $"Event moved to ({nx},{ny}).";
}
finally
{
    _draggingEvent = false;
    _dragEventId = null;
    _dragPreviewX = -1;
    _dragPreviewY = -1;

    HideDragOverlays();
    _dragAutoScrollTimer.Stop();

    if (Mouse.Captured == MapCanvas)
        MapCanvas.ReleaseMouseCapture();

    RequestViewportRender();

```

```
}  
}
```

```
private void UpdateDragViewportPoint(Point mouseViewport)
```

```
{  
    _dragLastViewportX = mouseViewport.X;  
    _dragLastViewportY = mouseViewport.Y;
```

```
    if (_draggingEvent)
```

```
    {  
        if (!_dragAutoScrollTimer.IsEnabled)  
            _dragAutoScrollTimer.Start();
```

```
    }
```

```
    else
```

```
    {  
        _dragAutoScrollTimer.Stop();  
    }
```

```
}
```

```
private void TickDragAutoScroll()
```

```
{  
    if (!_draggingEvent) { _dragAutoScrollTimer.Stop(); return; }  
    if (_currentMap is null) { _dragAutoScrollTimer.Stop(); return; }
```

```
    double vw = MapScroll.ViewportWidth;
```

```
    double vh = MapScroll.ViewportHeight;
```

```
    if (vw <= 1 || vh <= 1) return;
```

```

double dx = 0;

double dy = 0;

if (_dragLastViewportX < DragAutoScrollMargin) dx = -DragAutoScrollSpeed;
else if (_dragLastViewportX > vw - DragAutoScrollMargin) dx =
+DragAutoScrollSpeed;

if (_dragLastViewportY < DragAutoScrollMargin) dy = -DragAutoScrollSpeed;
else if (_dragLastViewportY > vh - DragAutoScrollMargin) dy =
+DragAutoScrollSpeed;

if (dx == 0 && dy == 0) return;

double newX = MapScroll.HorizontalOffset + dx;
double newY = MapScroll.VerticalOffset + dy;

if (newX < 0) newX = 0;
if (newY < 0) newY = 0;

MapScroll.ScrollToHorizontalOffset(newX);
MapScroll.ScrollToVerticalOffset(newY);

var mouseWorld = Mouse.GetPosition(MapCanvas);
_dragMouseWorldX = mouseWorld.X;
_dragMouseWorldY = mouseWorld.Y;
UpdateDragOverlays();
}

private MapEventDto? GetSelectedEvent()

```



```

{
    var map = _currentMap;
    if (map is null) return null;
    if (string.IsNullOrEmpty(_selectedEventId)) return null;
    return map.Events.FirstOrDefault(ev => ev.EventId == _selectedEventId);
}

```

```

// -----

```

```

// Event panel + event list ops

```

```

// -----

```

```

private void UpdateEventPanel(MapEventDto? ev)

```

```

{

```

```

    if (ev is null)

```

```

    {

```

```

        EventSelectedText.Text = "No event selected";

```

```

        EventNameBox.Text = "";

```

```

        EventTypeBox.SelectedIndex = 0;

```

```

        EventNameBox.IsEnabled = false;

```

```

        EventTypeBox.IsEnabled = false;

```

```

        DeleteEventButton.IsEnabled = false;

```

```

        WarpFieldsPanel.Visibility = Visibility.Collapsed;

```

```

        SetWarpDestMapUiValue("");

```

```

        WarpDestXBox.Text = "";

```

```

        WarpDestYBox.Text = "";

```

```

        WarpDestMapBox.IsEnabled = false;

```

```

        WarpDestXBox.IsEnabled = false;

```

```

WarpDestYBox.IsEnabled = false;

RefreshParamsList(null);

return;
}

EventSelectedText.Text = $"Selected @ ({ev.X},{ev.Y}) id={ev.EventId}";
EventNameBox.Text = ev.Name;

var types = ((IEnumerable<string>)EventTypeBox.ItemsSource!).ToList();
var idx = types.IndexOf(ev.Type);
EventTypeBox.SelectedIndex = idx >= 0 ? idx : 0;

EventNameBox.IsEnabled = true;
EventTypeBox.IsEnabled = true;
DeleteEventButton.IsEnabled = true;

bool isWarp = string.Equals(ev.Type, "Warp", StringComparison.OrdinalIgnoreCase);
WarpFieldsPanel.Visibility = isWarp ? Visibility.Visible : Visibility.Collapsed;

WarpDestMapBox.IsEnabled = isWarp;
WarpDestXBox.IsEnabled = isWarp;
WarpDestYBox.IsEnabled = isWarp;

if (isWarp)
    RefreshAvailableMaps(preserveCurrentSelection: false);

if (isWarp)

```

```

{
    var p = ev.Parameters ?? new Dictionary<string, string>();
    SetWarpDestMapUiValue(p.TryGetValue("destMap", out var dm) ? dm : "");
    WarpDestXBox.Text = p.TryGetValue("destX", out var dx) ? dx : "0";
    WarpDestYBox.Text = p.TryGetValue("destY", out var dy) ? dy : "0";
}
else
{
    SetWarpDestMapUiValue("");
    WarpDestXBox.Text = "";
    WarpDestYBox.Text = "";
}

RefreshParamsList(ev);
}

```

```

private void AddEvent(MapEventDto ev)
{
    if (_currentMap is null) return;
    var list = _currentMap.Events.ToList();
    list.Add(ev);
    _currentMap = _currentMap with { Events = list };
}

```

```

private void DeleteEvent(MapEventDto ev)
{
    if (_currentMap is null) return;
    var list = _currentMap.Events.Where(x => x.EventId != ev.EventId).ToList();
}

```

```
    _currentMap = _currentMap with { Events = list };  
}
```

```
private void ReplaceEvent(MapEventDto oldEv, MapEventDto newEv)  
{  
    if (_currentMap is null) return;  
    var list = _currentMap.Events.ToList();  
    var idx = list.FindIndex(x => x.EventId == oldEv.EventId);  
    if (idx < 0) return;  
    list[idx] = newEv;  
    _currentMap = _currentMap with { Events = list };  
}
```

```
private void OnDeleteEventClick(object sender, RoutedEventArgs e)  
{  
    var ev = GetSelectedEvent();  
    if (ev is null) return;  
  
    ExecuteCommand(new DeleteEventCommand(this, ev));  
  
    _selectedEventId = null;  
    _selectedCommandIndex = -1;  
    UpdateEventPanel(null);  
    RefreshCommandList(null);  
    UpdateCommandEditor(null);  
  
    RequestViewportRender();  
}
```

```

private void OnEventNameLostFocus(object sender, RoutedEventArgs e)
{
    var ev = GetSelectedEvent();
    if (ev is null) return;

    var newName = (EventNameBox.Text ?? "").Trim();
    if (string.IsNullOrEmpty(newName)) newName = "Event";
    if (newName == ev.Name) return;

    var updated = ev with { Name = newName };
    ExecuteCommand(new ReplaceEventCommand(this, ev, updated));
    UpdateEventPanel(updated);
    RefreshCommandList(updated);
}

```

```

private void OnEventTypeChanged(object sender, SelectionChangedEventArgs e)
{
    if (!IsLoaded) return;

    var ev = GetSelectedEvent();
    if (ev is null) return;

    var type = (EventTypeBox.SelectedItem as string) ?? "Trigger";
    if (type == ev.Type) return;

    var newParams = ev.Parameters is null
        ? new Dictionary<string, string>()

```

```

        : new Dictionary<string, string>(ev.Parameters);

        if (string.Equals(type, "Warp", StringComparison.OrdinalIgnoreCase))
        {
            if (!newParams.ContainsKey("destMap")) newParams["destMap"] =
"test_map_01";

            if (!newParams.ContainsKey("destX")) newParams["destX"] = "0";

            if (!newParams.ContainsKey("destY")) newParams["destY"] = "0";

        }

        var updated = ev with { Type = type, Parameters = newParams };
        ExecuteCommand(new ReplaceEventCommand(this, ev, updated));
        _selectedEventId = updated.EventId;

        UpdateEventPanel(updated);
        RefreshCommandList(updated);
        UpdateCommandEditor(null);
        _selectedCommandIndex = -1;

        RequestViewportRender();
    }

    private void OnWarpFieldsLostFocus(object sender, RoutedEventArgs e)
    {
        var ev = GetSelectedEvent();

        if (ev is null) return;

        if (!string.Equals(ev.Type, "Warp", StringComparison.OrdinalIgnoreCase)) return;

```

```

var destMap = GetWarpDestMapUiValue();
if (string.IsNullOrEmpty(destMap))
{
    StatusText.Text = "Warp destMap cannot be empty.";
    UpdateEventPanel(ev);
    return;
}

```

```

var destPathCheck = IOPath.Combine(MapsFolder, destMap +
MapFormat.DefaultExtension);
if (!File.Exists(destPathCheck))
{
    StatusText.Text = $"Warp destMap '{destMap}' does not exist in the maps folder.";
    UpdateEventPanel(ev);
    return;
}

```

```

if (!int.TryParse((WarpDestXBox.Text ?? "").Trim(), out var destX) ||
    !int.TryParse((WarpDestYBox.Text ?? "").Trim(), out var destY))
{
    StatusText.Text = "Warp destX/destY must be whole numbers.";
    UpdateEventPanel(ev);
    return;
}

```

```

var newParams = ev.Parameters is null

```

```

        ? new Dictionary<string, string>()
        : new Dictionary<string, string>(ev.Parameters);

newParams["destMap"] = destMap;
newParams["destX"] = destX.ToString();
newParams["destY"] = destY.ToString();

var updated = ev with { Parameters = newParams };
ExecuteCommand(new ReplaceEventCommand(this, ev, updated));
UpdateEventPanel(updated);
RefreshCommandList(updated);
}

// -----
// Parameters editor
// -----

private void RefreshParamsList(MapEventDto? ev)
{
    ParamsList.Items.Clear();
    ParamKeyBox.Text = "";
    ParamValueBox.Text = "";

    if (ev is null) return;

    var dict = ev.Parameters ?? new Dictionary<string, string>();
    foreach (var kv in dict.OrderBy(k => k.Key, StringComparer.OrdinalIgnoreCase))
        ParamsList.Items.Add($"{kv.Key} = {kv.Value}");
}

```



```

private void OnParamSelectionChanged(object sender, SelectionChangedEventArgs
e)
{
    var ev = GetSelectedEvent();
    if (ev is null) return;

    if (ParamsList.SelectedItem is not string row) return;

    var cut = row.IndexOf("=");
    if (cut <= 0) return;

    ParamKeyBox.Text = row[..cut].Trim();
    ParamValueBox.Text = row[(cut + 1)..].Trim();
}

```

```

private void OnAddParamClick(object sender, RoutedEventArgs e)
{
    var ev = GetSelectedEvent();
    if (ev is null) { StatusText.Text = "Select an event first."; return; }

    var key = (ParamKeyBox.Text ?? "").Trim();
    var val = ParamValueBox.Text ?? "";

    if (key.Length == 0)
    {
        StatusText.Text = "Parameter key cannot be empty.";
        return;
    }
}

```

```
}
```

```
var dict = ev.Parameters is null
```

```
    ? new Dictionary<string, string>()
```

```
    : new Dictionary<string, string>(ev.Parameters);
```

```
dict[key] = val;
```

```
var updated = ev with { Parameters = dict };
```

```
ExecuteCommand(new ReplaceEventCommand(this, ev, updated));
```

```
_selectedEventId = updated.EventId;
```

```
UpdateEventPanel(updated);
```

```
RefreshCommandList(updated);
```

```
UpdateCommandEditor(null);
```

```
StatusText.Text = $"Param set: {key} = {val}";
```

```
}
```

```
private void OnDeleteParamClick(object sender, RoutedEventArgs e)
```

```
{
```

```
    var ev = GetSelectedEvent();
```

```
    if (ev is null) return;
```

```
    if (ParamsList.SelectedItem is not string row || row.Length == 0)
```

```
    {
```

```
        StatusText.Text = "Select a parameter row to delete.";
```

```
        return;
```

```

    }

    var cut = row.IndexOf("=");
    if (cut <= 0) return;

    var key = row[..cut].Trim();

    var dict = ev.Parameters is null
        ? new Dictionary<string, string>()
        : new Dictionary<string, string>(ev.Parameters);

    if (!dict.Remove(key))
    {
        StatusText.Text = "Parameter not found.";
        return;
    }

    var updated = ev with { Parameters = dict };
    ExecuteCommand(new ReplaceEventCommand(this, ev, updated));

    _selectedEventId = updated.EventId;
    UpdateEventPanel(updated);

    StatusText.Text = $"Param removed: {key}";
}

// -----
// Command list + editor

```

```
// -----
private void RefreshCommandList(MapEventDto? ev)
{
    _updatingCommandUi = true;

    try
    {
        EventCommandList.Items.Clear();

        _selectedCommandIndex = -1;
        EventCommandList.SelectedIndex = -1;

        if (ev is null) return;

        var cmds = ev.Commands ?? Array.Empty<EventCommandDto>();
        for (int i = 0; i < cmds.Count; i++)
            EventCommandList.Items.Add(FormatCommandRow(i, cmds[i]));
    }
    finally
    {
        _updatingCommandUi = false;
    }
}

private static string FormatCommandRow(int i, EventCommandDto cmd)
{
    var c = (cmd.Command ?? "").Trim();

    if (c == "ShowText")
    {
```

```

cmd.Args.TryGetValue("text", out var t);
t ??= "";
t = t.Replace("\r", " ").Replace("\n", " ");
if (t.Length > 30) t = t[..30] + "...";
return $"{i:00}: ShowText \"{t}\"";
}

```

```

if (c == "SetFlag")
{
    cmd.Args.TryGetValue("flag", out var f);
    cmd.Args.TryGetValue("value", out var v);
    return $"{i:00}: SetFlag {f}={v}";
}

```

```

return $"{i:00}: {c}";
}

```

```

private void OnCommandSelectionChanged(object sender,
SelectionChangedEventArgs e)

```

```

{
    if (_updatingCommandUi) return;

    var ev = GetSelectedEvent();
    if (ev is null)
    {
        UpdateCommandEditor(null);
        return;
    }
}

```

```

        _selectedCommandIndex = EventCommandList.SelectedIndex;
        if (_selectedCommandIndex < 0)
        {
            UpdateCommandEditor(null);
            return;
        }

        var cmds = ev.Commands ?? Array.Empty<EventCommandDto>();
        if (_selectedCommandIndex >= cmds.Count)
        {
            UpdateCommandEditor(null);
            return;
        }

        UpdateCommandEditor(cmds[_selectedCommandIndex]);
    }

```

```

private void UpdateCommandEditor(EventCommandDto? cmd)
{
    _updatingCommandUi = true;
    try
    {
        if (cmd is null)
        {
            CommandEditorHeader.Text = "No command selected";
            ShowTextEditorPanel.Visibility = Visibility.Collapsed;
            SetFlagEditorPanel.Visibility = Visibility.Collapsed;
        }
    }
}

```

```
        return;
    }

    var c = (cmd.Command ?? "").Trim();
    CommandEditorHeader.Text = $"Editing: {c}";

    if (c == "ShowText")
    {
        ShowTextEditorPanel.Visibility = Visibility.Visible;
        SetFlagEditorPanel.Visibility = Visibility.Collapsed;

        cmd.Args.TryGetValue("text", out var t);
        CmdTextBox.Text = t ?? "";

        return;
    }

    if (c == "SetFlag")
    {
        ShowTextEditorPanel.Visibility = Visibility.Collapsed;
        SetFlagEditorPanel.Visibility = Visibility.Visible;

        cmd.Args.TryGetValue("flag", out var f);
        cmd.Args.TryGetValue("value", out var v);
        CmdFlagNameBox.Text = f ?? "";
        CmdFlagValueBox.Text = v ?? "";

        return;
    }
```

```

        ShowTextEditorPanel.Visibility = Visibility.Collapsed;
        SetFlagEditorPanel.Visibility = Visibility.Collapsed;
    }
    finally
    {
        _updatingCommandUi = false;
    }
}

private void OnAddShowTextClick(object sender, RoutedEventArgs e)
{
    var ev = GetSelectedEvent();
    if (ev is null) { StatusText.Text = "Select an event first."; return; }

    var cmd = new EventCommandDto("ShowText", new Dictionary<string, string> {
["text"] = "Hello!" });
    var updated = AppendCommand(ev, cmd);
    ExecuteCommand(new ReplaceEventCommand(this, ev, updated));

    RefreshCommandList(updated);

    _updatingCommandUi = true;
    try { EventCommandList.SelectedIndex = (updated.Commands?.Count ?? 1) - 1; }
    finally { _updatingCommandUi = false; }

    _selectedCommandIndex = EventCommandList.SelectedIndex;
    UpdateCommandEditor(updated.Commands![_selectedCommandIndex]);
    RequestViewportRender();
}

```



```
}
```

```
private void OnAddSetFlagClick(object sender, RoutedEventArgs e)
{
    var ev = GetSelectedEvent();
    if (ev is null) { StatusText.Text = "Select an event first."; return; }

    var cmd = new EventCommandDto("SetFlag", new Dictionary<string, string>
    {
        ["flag"] = "has_key",
        ["value"] = "true"
    });

    var updated = AppendCommand(ev, cmd);
    ExecuteCommand(new ReplaceEventCommand(this, ev, updated));

    RefreshCommandList(updated);

    _updatingCommandUi = true;
    try { EventCommandList.SelectedIndex = (updated.Commands?.Count ?? 1) - 1; }
    finally { _updatingCommandUi = false; }

    _selectedCommandIndex = EventCommandList.SelectedIndex;
    UpdateCommandEditor(updated.Commands![_selectedCommandIndex]);
    RequestViewportRender();
}
```

```

private static MapEventDto AppendCommand(MapEventDto ev, EventCommandDto
cmd)
{
    var list = (ev.Commands ?? Array.Empty<EventCommandDto>()).ToList();
    list.Add(cmd);
    return ev with { Commands = list };
}

```

```

private void OnDeleteCommandClick(object sender, RoutedEventArgs e)
{
    var ev = GetSelectedEvent();
    if (ev is null) return;

    _selectedCommandIndex = EventCommandList.SelectedIndex;

    if (_selectedCommandIndex < 0) { StatusText.Text = "Select a command to delete.";
return; }

```

```

    var list = (ev.Commands ?? Array.Empty<EventCommandDto>()).ToList();
    if (_selectedCommandIndex >= list.Count) return;

```

```

    list.RemoveAt(_selectedCommandIndex);

```

```

    var updated = ev with { Commands = list };
    ExecuteCommand(new ReplaceEventCommand(this, ev, updated));

```

```

    RefreshCommandList(updated);

```

```

    UpdateCommandEditor(null);

```

```

    _selectedCommandIndex = -1;

```

```

}

```

```

private void OnMoveCommandUpClick(object sender, RoutedEventArgs e) =>
MoveCommand(-1);

private void OnMoveCommandDownClick(object sender, RoutedEventArgs e) =>
MoveCommand(+1);


private void MoveCommand(int delta)
{
    var ev = GetSelectedEvent();
    if (ev is null) return;

    var list = (ev.Commands ?? Array.Empty<EventCommandDto>()).ToList();
    _selectedCommandIndex = EventCommandList.SelectedIndex;
    if (_selectedCommandIndex < 0 || _selectedCommandIndex >= list.Count) return;

    int j = _selectedCommandIndex + delta;
    if (j < 0 || j >= list.Count) return;

    (list[_selectedCommandIndex], list[j]) = (list[j], list[_selectedCommandIndex]);

    var updated = ev with { Commands = list };
    ExecuteCommand(new ReplaceEventCommand(this, ev, updated));

    RefreshCommandList(updated);

    _updatingCommandUi = true;
    try { EventCommandList.SelectedIndex = j; }
    finally { _updatingCommandUi = false; }
}

```

```
    _selectedCommandIndex = j;  
    UpdateCommandEditor(updated.Commands![_selectedCommandIndex]);  
}
```

```
private void OnCommandFieldsLostFocus(object sender, RoutedEventArgs e)  
{
```

```
    if (_updatingCommandUi) return;
```

```
    var ev = GetSelectedEvent();
```

```
    if (ev is null) return;
```

```
    int idx = EventCommandList.SelectedIndex;
```

```
    if (idx < 0) return;
```

```
    var list = (ev.Commands ?? Array.Empty<EventCommandDto>()).ToList();
```

```
    if (list.Count == 0) return;
```

```
    if (idx >= list.Count) return;
```

```
    var old = list[idx];
```

```
    var c = (old.Command ?? "").Trim();
```

```
    if (c == "ShowText")
```

```
    {
```

```
        var text = CmdTextBox.Text ?? "";
```

```
        list[idx] = new EventCommandDto("ShowText",  
            new Dictionary<string, string> { ["text"] = text });
```

```
    }
```

```
    else if (c == "SetFlag")
```

```

{
    var flag = (CmdFlagNameBox.Text ?? "").Trim();
    var value = CmdFlagValueBox.Text ?? "";

    if (flag.Length == 0)
    {
        StatusText.Text = "SetFlag: flag cannot be empty.";
        return;
    }

    list[idx] = new EventCommandDto("SetFlag",
        new Dictionary<string, string>
        {
            ["flag"] = flag,
            ["value"] = value
        });
    }
    else
    {
        return;
    }

    var updatedEv = ev with { Commands = list };

    _updatingCommandUi = true;

    try
    {
        ExecuteCommand(new ReplaceEventCommand(this, ev, updatedEv));
    }
}

```

```

RefreshCommandList(updatedEv);

if (idx >= 0 && idx < (updatedEv.Commands?.Count ?? 0))
{
    EventCommandList.SelectedIndex = idx;
    UpdateCommandEditor(updatedEv.Commands![idx]);
}
else
{
    EventCommandList.SelectedIndex = -1;
    UpdateCommandEditor(null);
}
}
finally
{
    _updatingCommandUi = false;
}

RequestViewportRender();
}

// -----
// Undo/redo + persistence
// -----

private void ExecuteCommand(IEditCommand cmd)
{
    cmd.Execute();
}

```

```
    _undo.Push(cmd);  
    _redo.Clear();  
  
    PersistMap();  
    RequestViewportRender();  
}
```

```
private void Undo()  
{  
    if (_undo.Count == 0) return;  
    var cmd = _undo.Pop();  
    cmd.Undo();  
    _redo.Push(cmd);  
  
    PersistMap();  
    RequestViewportRender();  
}
```

```
private void Redo()  
{  
    if (_redo.Count == 0) return;  
    var cmd = _redo.Pop();  
    cmd.Execute();  
    _undo.Push(cmd);  
  
    PersistMap();  
    RequestViewportRender();  
}
```

```

private void PersistMap()
{
    if (_currentMap is null || string.IsNullOrEmpty(_currentMapPath))
        return;

    if (!IsSampleMapLoaded() && !_manualSavePerformed)
    {
        StatusText.Text = $"Not saved yet — press Save to enable autosave.
(Undo:{_undo.Count} Redo:{_redo.Count})";

        return;
    }

    MapSerializer.SaveToFile(_currentMapPath, _currentMap);

    StatusText.Text = $"Saved: {_currentMapPath} (Undo:{_undo.Count}
Redo:{_redo.Count})";
}

// -----
// Undo/redo command implementations
// -----

private sealed class PaintIntCellCommand : IEditCommand
{
    private readonly MainWindow _w;

    private readonly LayerKind _kind;

    private readonly int _idx;

    private readonly int _oldValue;

    private readonly int _newValue;

```



```
    public PaintIntCellCommand(MainWindow w, LayerKind kind, int idx, int oldValue,
int newValue)
```

```
    {
        _w = w; _kind = kind; _idx = idx; _oldValue = oldValue; _newValue = newValue;
    }
```

```
    public void Execute() => _w.SetLayerCell(_kind, _idx, _newValue);
```

```
    public void Undo() => _w.SetLayerCell(_kind, _idx, _oldValue);
```

```
}
```

```
private sealed class AddEventCommand : IEditCommand
```

```
{
```

```
    private readonly MainWindow _w;
```

```
    private readonly MapEventDto _ev;
```

```
    public AddEventCommand(MainWindow w, MapEventDto ev) { _w = w; _ev = ev; }
```

```
    public void Execute() => _w.AddEvent(_ev);
```

```
    public void Undo() => _w.DeleteEvent(_ev);
```

```
}
```

```
private sealed class DeleteEventCommand : IEditCommand
```

```
{
```

```
    private readonly MainWindow _w;
```

```
    private readonly MapEventDto _ev;
```

```
    public DeleteEventCommand(MainWindow w, MapEventDto ev) { _w = w; _ev = ev; }
```

```
    public void Execute() => _w.DeleteEvent(_ev);
```

```
    public void Undo() => _w.AddEvent(_ev);
```

```
}
```

```

private sealed class ReplaceEventCommand : IEditCommand
{
    private readonly MainWindow _w;
    private readonly MapEventDto _oldEv;
    private readonly MapEventDto _newEv;

    public ReplaceEventCommand(MainWindow w, MapEventDto oldEv, MapEventDto
newEv) { _w = w; _oldEv = oldEv; _newEv = newEv; }

    public void Execute() => _w.ReplaceEvent(_oldEv, _newEv);
    public void Undo() => _w.ReplaceEvent(_newEv, _oldEv);
}

```

```

private sealed class SwapEventsCommand : IEditCommand
{
    private readonly MainWindow _w;
    private readonly MapEventDto _oldA;
    private readonly MapEventDto _newA;
    private readonly MapEventDto _oldB;
    private readonly MapEventDto _newB;

    public SwapEventsCommand(MainWindow w, MapEventDto oldA, MapEventDto
newA, MapEventDto oldB, MapEventDto newB)
    {
        _w = w;
        _oldA = oldA; _newA = newA;
        _oldB = oldB; _newB = newB;
    }

    public void Execute()
    {

```

```

        _w.ReplaceEvent(_oldA, _newA);
        _w.ReplaceEvent(_oldB, _newB);
    }

    public void Undo()
    {
        _w.ReplaceEvent(_newA, _oldA);
        _w.ReplaceEvent(_newB, _oldB);
    }
}

private sealed class SwapCellContentsCommand : IEditCommand
{
    private readonly MainWindow _w;
    private readonly int _ax, _ay;
    private readonly int _bx, _by;

    private readonly int _aBg, _bBg;
    private readonly int _aCol, _bCol;

    private readonly MapEventDto? _aEv;
    private readonly MapEventDto? _bEv;

    public SwapCellContentsCommand(MainWindow w, int ax, int ay, int bx, int by)
    {
        _w = w;
        _ax = ax; _ay = ay;
        _bx = bx; _by = by;
    }
}

```

```

var map = _w._currentMap!;
var bg = _w.GetBackgroundLayer(!);
var col = _w.GetCollisionLayer();

int aldx = _ay * map.Width + _ax;
int bldx = _by * map.Width + _bx;

_aBg = bg.Tiles[aldx];
_bBg = bg.Tiles[bldx];

_aCol = (col is null) ? 0 : col.Tiles[aldx];
_bCol = (col is null) ? 0 : col.Tiles[bldx];

_aEv = _w.GetEventAt(_ax, _ay);
_bEv = _w.GetEventAt(_bx, _by);
}

public void Execute() => _w.ApplySwapCellContents(_ax, _ay, _bx, _by);
public void Undo() => _w.ApplySwapCellContents(_ax, _ay, _bx, _by);
}

private void ApplySwapCellContents(int ax, int ay, int bx, int by)
{
var map = _currentMap;
if (map is null) return;

int aldx = ay * map.Width + ax;

```

```
int bldx = by * map.Width + bx;
```

```
var bg = GetBackgroundLayer();
```

```
if (bg is null) return;
```

```
var col = GetCollisionLayer();
```

```
(bg.Tiles[aldx], bg.Tiles[bldx]) = (bg.Tiles[bldx], bg.Tiles[aldx]);
```

```
if (col is not null && col.Tiles.Length == map.Width * map.Height)
```

```
    (col.Tiles[aldx], col.Tiles[bldx]) = (col.Tiles[bldx], col.Tiles[aldx]);
```

```
var aEv = GetEventAt(ax, ay);
```

```
var bEv = GetEventAt(bx, by);
```

```
if (aEv is not null && bEv is not null)
```

```
{
```

```
    var newA = aEv with { X = bx, Y = by };
```

```
    var newB = bEv with { X = ax, Y = ay };
```

```
    ReplaceEvent(aEv, newA);
```

```
    ReplaceEvent(bEv, newB);
```

```
    if (_selectedEventId == aEv.EventId) _selectedEventId = newA.EventId;
```

```
    else if (_selectedEventId == bEv.EventId) _selectedEventId = newB.EventId;
```

```
}
```

```
else if (aEv is not null)
```

```
{
```

```
    var newA = aEv with { X = bx, Y = by };
```

```

        ReplaceEvent(aEv, newA);
        if (_selectedEventId == aEv.EventId) _selectedEventId = newA.EventId;
    }
    else if (bEv is not null)
    {
        var newB = bEv with { X = ax, Y = ay };
        ReplaceEvent(bEv, newB);
        if (_selectedEventId == bEv.EventId) _selectedEventId = newB.EventId;
    }
}

```

```

private void SetLayerCell(LayerKind kind, int idx, int value)
{
    var map = _currentMap;
    if (map is null) return;

    var layer = map.Layers.FirstOrDefault(l => l.Kind == kind);
    if (layer is null) return;
    if (idx < 0 || idx >= layer.Tiles.Length) return;

    layer.Tiles[idx] = value;
}

```

```

// -----
// Arrow-key: move cell contents (tile + collision + event) together
// -----

private void TryMoveSelectedCell(int dx, int dy)
{

```

```

var map = _currentMap;

if (map is null) return;

if (_playtestActive)
    return;

// Use cursor tile if set, else fallback to hover
int x = _cursorTileX >= 0 ? _cursorTileX : _hoverTileX;
int y = _cursorTileY >= 0 ? _cursorTileY : _hoverTileY;

if (x < 0 || y < 0 || x >= map.Width || y >= map.Height)
    return;

int nx = x + dx;
int ny = y + dy;

if (nx < 0 || ny < 0 || nx >= map.Width || ny >= map.Height)
    return;

ExecuteCommand(new SwapCellContentsCommand(this, x, y, nx, ny));
ShowSwapHint(x, y, nx, ny);

_cursorTileX = nx;
_cursorTileY = ny;

var sel = GetSelectedEvent();
UpdateEventPanel(sel);
RefreshCommandList(sel);

```

```

        UpdateCommandEditor(null);
    }

    // -----
    // Keyboard handling
    // -----

    private void OnMainWindowPreviewKeyDown(object sender, KeyEventArgs e)
    {
        if ((Keyboard.Modifiers & ModifierKeys.Control) != 0)
        {
            if (e.Key == Key.Z) { Undo(); e.Handled = true; return; }
            if (e.Key == Key.Y) { Redo(); e.Handled = true; return; }
            if (e.Key == Key.D)
        {
            // Duplicate selected event (events mode only)
            if (_playtestActive) { e.Handled = true; return; }
            if (_mode != EditMode.Events) { e.Handled = true; return; }

            var src = GetSelectedEvent();
            if (src is null) { e.Handled = true; return; }

            // Prefer cursor tile, else hover tile, else next to the event
            int tx = _cursorTileX >= 0 ? _cursorTileX : (_hoverTileX >= 0 ? _hoverTileX : src.X);
            int ty = _cursorTileY >= 0 ? _cursorTileY : (_hoverTileY >= 0 ? _hoverTileY : src.Y);

            if (!TryFindFreeTileNear(tx, ty, out var fx, out var fy))
            {
                StatusText.Text = "No free tile nearby to duplicate into.";
            }
        }
    }

```



```

        e.Handled = true;

        return;
    }

    var copy = CloneEventTo(src, fx, fy);
    ExecuteCommand(new AddEventCommand(this, copy));

    _selectedEventId = copy.EventId;
    UpdateEventPanel(copy);
    RefreshCommandList(copy);
    UpdateCommandEditor(null);
    RequestViewportRender();

    StatusText.Text = $"Duplicated event to ({fx},{fy}).";
    e.Handled = true;
    return;
}

}

// Arrow keys: if not playtesting, move the selected cell contents
(tile+collision+event) together.
if (!_playtestActive && Keyboard.Modifiers == ModifierKeys.None)
{
    switch (e.Key)
    {
        case Key.Up: TryMoveSelectedCell(0, -1); e.Handled = true; return;
        case Key.Down: TryMoveSelectedCell(0, 1); e.Handled = true; return;
    }
}

```

```
        case Key.Left: TryMoveSelectedCell(-1, 0); e.Handled = true; return;
        case Key.Right: TryMoveSelectedCell(1, 0); e.Handled = true; return;
    }
}
```

```
if (e.Key == Key.Delete)
{
    DeleteAtCurrentMode();
    e.Handled = true;
    return;
}
```

```
if (e.Key == Key.F5) { TogglePlaytest(); e.Handled = true; return; }
```

```
if (_playtestActive)
{
    if (e.Key == Key.Escape) { ExitPlaytest(); e.Handled = true; return; }
```

```
    if (e.Key == Key.E || e.Key == Key.Enter || e.Key == Key.Space)
    {
        InteractAtPlayer();
        e.Handled = true;
        return;
    }
```

```
    switch (e.Key)
    {
        case Key.Up:
```

```

        case Key.W: TryMovePlayer(0, -1); e.Handled = true; return;
        case Key.Down:
        case Key.S: TryMovePlayer(0, 1); e.Handled = true; return;
        case Key.Left:
        case Key.A: TryMovePlayer(-1, 0); e.Handled = true; return;
        case Key.Right:
        case Key.D: TryMovePlayer(1, 0); e.Handled = true; return;
    }
}

// -----
// Zoom
// -----

private void ApplyZoom()
{
    _zoom = Math.Clamp(_zoom, ZoomMin, ZoomMax);
    MapCanvas.LayoutTransform = new ScaleTransform(_zoom, _zoom);
    ZoomText.Text = $"Zoom: {(_zoom * 100):0}%";
}

private void ZoomAtViewportPoint(Point viewportPoint, double newZoom)
{
    newZoom = Math.Clamp(newZoom, ZoomMin, ZoomMax);
    if (Math.Abs(newZoom - _zoom) < 0.0001) return;

    double oldZoom = _zoom;

```

```
double contentX = MapScroll.HorizontalOffset + viewportPoint.X;
```

```
double contentY = MapScroll.VerticalOffset + viewportPoint.Y;
```

```
_zoom = newZoom;
```

```
ApplyZoom();
```

```
double ratio = newZoom / oldZoom;
```

```
MapScroll.Dispatcher.BeginInvoke(DispatcherPriority.Background, new Action(() =>
```

```
{
```

```
    double newOffsetX = contentX * ratio - viewportPoint.X;
```

```
    double newOffsetY = contentY * ratio - viewportPoint.Y;
```

```
    if (newOffsetX < 0) newOffsetX = 0;
```

```
    if (newOffsetY < 0) newOffsetY = 0;
```

```
    MapScroll.ScrollToHorizontalOffset(newOffsetX);
```

```
    MapScroll.ScrollToVerticalOffset(newOffsetY);
```

```
    RequestViewportRender();
```

```
    }));
```

```
}
```

```
private void OnZoomInClick(object sender, RoutedEventArgs e)
```

```
{
```

```
    var viewportCenter = new Point(MapScroll.ViewportWidth / 2.0,  
MapScroll.ViewportHeight / 2.0);
```

```
    ZoomAtViewportPoint(viewportCenter, _zoom * ZoomStep);
```

```
}
```

```
private void OnZoomOutClick(object sender, RoutedEventArgs e)
```

```
{
```

```
    var viewportCenter = new Point(MapScroll.ViewportWidth / 2.0,  
MapScroll.ViewportHeight / 2.0);
```

```
    ZoomAtViewportPoint(viewportCenter, _zoom / ZoomStep);
```

```
}
```

```
private void OnZoomResetClick(object sender, RoutedEventArgs e)
```

```
{
```

```
    var viewportCenter = new Point(MapScroll.ViewportWidth / 2.0,  
MapScroll.ViewportHeight / 2.0);
```

```
    ZoomAtViewportPoint(viewportCenter, 1.0);
```

```
}
```

```
private void OnScrollViewerMouseWheel(object sender, MouseWheelEventArgs e)
```

```
{
```

```
    if ((Keyboard.Modifiers & ModifierKeys.Control) == 0) return;
```

```
    e.Handled = true;
```

```
    var viewportPos = e.GetPosition(MapScroll);
```

```
    double newZoom = _zoom;
```

```
    if (e.Delta > 0) newZoom *= ZoomStep;
```

```
    else newZoom /= ZoomStep;
```

```
    ZoomAtViewportPoint(viewportPos, newZoom);
```

```
}
```

```

// -----
// Command interface
// -----

private interface IEditCommand
{
    void Execute();

    void Undo();
}
}

Vitriol.Editor\Vitriol.Editor.csproj
<Project Sdk="Microsoft.NET.Sdk">

    <ItemGroup>

        <ProjectReference Include="..\Vitriol.Shared\Vitriol.Shared.csproj" />
        <ProjectReference Include="..\Vitriol.Data\Vitriol.Data.csproj" />
        <ProjectReference Include="..\Vitriol.Engine\Vitriol.Engine.csproj" />
    </ItemGroup>

    <PropertyGroup>
        <OutputType>WinExe</OutputType>
        <TargetFramework>net9.0-windows</TargetFramework>
        <Nullable>enable</Nullable>
        <ImplicitUsings>enable</ImplicitUsings>
        <UseWPF>true</UseWPF>
    </PropertyGroup>

</Project>

```

Vitriol.Engine

Vitriol.Engine\Maps

Vitriol.Engine\Maps\MapFormat.cs

```
namespace Vitriol.Engine.Maps;
```

```
public static class MapFormat
```

```
{
```

```
    public const int CurrentVersion = 1;
```

```
    public const string DefaultExtension = ".vitmap.json";
```

```
}
```

Vitriol.Engine\Maps\MapSerializer.cs

```
using System.Text.Json;
```

```
using Vitriol.Shared.Maps;
```

```
namespace Vitriol.Engine.Maps;
```

```
public static class MapSerializer
```

```
{
```

```
    private static readonly JsonSerializerOptions Options = new()
```

```
    {
```

```
        WriteIndented = true,
```

```
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase
```

```
    };
```

```
    public static string ToJson(MapDto map)
```

```
        => JsonSerializer.Serialize(map, Options);
```

```
    public static MapDto FromJson(string json)
```

```
=> JsonSerializer.Deserialize<MapDto>(json, Options)
    ?? throw new InvalidOperationException("Failed to deserialize MapDto (null).");
```

```
public static void SaveToFile(string path, MapDto map)
{
    Directory.CreateDirectory(Path.GetDirectoryName(path)!);
    File.WriteAllText(path, ToJson(map));
}
```

```
public static MapDto LoadFromFile(string path)
    => FromJson(File.ReadAllText(path));
}
```

Vitriol.Engine\Class1.cs

```
namespace Vitriol.Engine;
```

```
public class Class1
```

```
{
```

```
}
```

Vitriol.Engine\Vitriol.Engine.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
<ItemGroup>
```

```
<ProjectReference Include="..\Vitriol.Shared\Vitriol.Shared.csproj" />
```

```
</ItemGroup>
```

```
<PropertyGroup>
```

```
<TargetFramework>net9.0</TargetFramework>
```



```
<ImplicitUsings>enable</ImplicitUsings>

<Nullable>enable</Nullable>

</PropertyGroup>

</Project>
```

Vitriol.Runtime

Vitriol.Runtime\App.xaml

```
<Application x:Class="Vitriol.Runtime.App"

    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

    xmlns:local="clr-namespace:Vitriol.Runtime"

    StartupUri="MainWindow.xaml">

    <Application.Resources>

        </Application.Resources>

    </Application>
```

Vitriol.Runtime\App.xaml.cs

```
using System.Configuration;

using System.Data;

using System.Windows;

namespace Vitriol.Runtime;

/// <summary>

/// Interaction logic for App.xaml

/// </summary>

public partial class App : Application
```

```
{  
}
```

Vitriol.Runtime\AssemblyInfo.cs

```
using System.Windows;
```

```
[assembly:ThemeInfo(  
    ResourceDictionaryLocation.None,    //where theme specific resource
```

```
    dictionaries are located  
  
    //(used if a resource is not found in the page,  
    // or application resource dictionaries)
```

```
    ResourceDictionaryLocation.SourceAssembly //where the generic resource  
    dictionary is located  
  
    //(used if a resource is not found in the page,  
    // app, or any theme specific resource dictionaries)
```

```
)]
```

Vitriol.Runtime\MainWindow.xaml

```
<Window x:Class="Vitriol.Runtime.MainWindow"
```

```
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
```

```
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
```

```
    Title="Vitriol Runtime"
```

```
    Width="1100" Height="700"
```

```
    Background="#111"
```

```
    KeyDown="OnWindowKeyDown"
```

```
    WindowStyle="None"
```

```
    ResizeMode="NoResize"
```

```
    WindowState="Maximized"
```

```
    WindowStartupLocation="CenterScreen">
```

```
<Grid Margin="12">
```

```
<Grid.RowDefinitions>

    <RowDefinition Height="Auto"/>

    <RowDefinition Height="*/>

</Grid.RowDefinitions>
```

```
<!-- HUD -->
```

```
<Border Grid.Row="0"

    BorderBrush="#444" BorderThickness="1"

    CornerRadius="6"

    Padding="10"

    Margin="0,0,0,10"

    Background="#1a1a1a">
```

```
<DockPanel LastChildFill="True">
```

```
    <Button Width="120"

        Height="30"

        Margin="0,0,10,0"

        Click="OnLoadMapClick"

        DockPanel.Dock="Left"

        Focusable="False"

        IsTabStop="False"

        IsDefault="False">
```

```
        Load Map
```

```
</Button>
```

```
<TextBlock Name="HudText"
```

```
    FontSize="14"
```

```
    Foreground="White"
```

```

        Text="Booting..."
        TextTrimming="CharacterEllipsis"
        VerticalAlignment="Center"/>
    </DockPanel>
</Border>

<!-- Viewport -->
<Border Grid.Row="1"
        BorderBrush="#444" BorderThickness="1"
        CornerRadius="6"
        Background="#000">

    <ScrollViewer Name="ViewScroll"
        HorizontalScrollBarVisibility="Auto"
        VerticalScrollBarVisibility="Auto"
        ScrollChanged="OnScrollChanged"
        Focusable="True">

        <Canvas Name="ViewCanvas"
            Background="#000"
            Focusable="True"/>

    </ScrollViewer>

</Border>
</Grid>
</Window>

```

Vitriol.Runtime\MainWindow.xaml.cs

```

using System;
using System.Collections.Concurrent;

```

```

using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Windows;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Controls;
using System.Windows.Threading;
using Microsoft.Win32;
using Vitriol.Engine.Maps;
using Vitriol.Shared.Maps;
using IOPath = System.IO.Path;
using System.Windows.Interop;

namespace Vitriol.Runtime;

public partial class MainWindow : Window
{
    // -----
    // Fullscreen lock
    // -----

    private bool _fullscreenApplying = false;

    private void ForceFullscreen()
    {
        if (_fullscreenApplying) return;
        _fullscreenApplying = true;
    }

```

```
try
{
    if (WindowState != WindowStyle.None) WindowStyle = WindowStyle.None;

    if (ResizeMode != ResizeMode.NoResize) ResizeMode = ResizeMode.NoResize;

    if (WindowState != WindowState.Maximized) WindowState =
WindowState.Maximized;
}

finally
{
    _fullscreenApplying = false;
}
}
```

```
private void HookFullscreenGuards()
{
    Loaded += (_, _) => ForceFullscreen();
    StateChanged += (_, _) => ForceFullscreen();
    SizeChanged += (_, _) => ForceFullscreen();
    Activated += (_, _) => ForceFullscreen();
}
```

```
protected override void OnSourceInitialized(EventArgs e)
{
    base.OnSourceInitialized(e);

    ForceFullscreen();
}
```

```

var hwnd = new WindowInteropHelper(this).Handle;
if (hwnd != IntPtr.Zero)
{
    const int GWL_STYLE = -16;
    const int WS_SYSMENU = 0x00080000;

    int style = NativeMethods.GetWindowLong(hwnd, GWL_STYLE);
    style &= ~WS_SYSMENU;
    NativeMethods.SetWindowLong(hwnd, GWL_STYLE, style);
}
}

private static class NativeMethods
{
    [System.Runtime.InteropServices.DllImport("user32.dll")]
    public static extern int GetWindowLong(IntPtr hWnd, int nIndex);

    [System.Runtime.InteropServices.DllImport("user32.dll")]
    public static extern int SetWindowLong(IntPtr hWnd, int nIndex, int dwNewLong);
}

// -----
// Project folders
// -----

private readonly string _projectRoot;
private string MapsFolder => IOPath.Combine(_projectRoot, "maps");
private string PngsFolder => IOPath.Combine(_projectRoot, "assets", "tilesets");

```

```

private MapDto? _map;

private string? _mapPath;


private const double DeadzoneFracX = 0.55;
private const double DeadzoneFracY = 0.28;


private int _playerX = 0;
private int _playerY = 0;

private readonly Dictionary<string, string> _flags =
new(StringComparer.OrdinalIgnoreCase);


// -----
// Tileset caches + compiled runtime layout (multi-tileset, global indices)
// -----

private readonly ConcurrentDictionary<string, BitmapSource> _tilesetBitmapCache =
new(StringComparer.OrdinalIgnoreCase);

private readonly ConcurrentDictionary<string, ConcurrentDictionary<int,
CroppedBitmap>> _tileCropCache = new(StringComparer.OrdinalIgnoreCase);


private readonly object _tilesetCompileLock = new();
private List<TilesetRuntime> _compiledTilesets = new();
private bool _tilesetsDirty = true;


private sealed record TilesetMeta(
    int PixelWidth,
    int PixelHeight,
    int TileWidth,
    int TileHeight,
    int Margin,

```



```
    int Spacing,  
    int TilesAcross,  
    int TilesDown,  
    int TileCount  
);
```

```
private sealed class TilesetRuntime
```

```
{  
    public required TilesetRefDto Ref { get; init; }  
    public required string FullPath { get; init; }  
    public required int StartIndexGlobal { get; init; }  
    public required int TileCount { get; init; }  
    public required int TilesAcross { get; init; }  
    public required int TileW { get; init; }  
    public required int TileH { get; init; }  
    public required int Margin { get; init; }  
    public required int Spacing { get; init; }  
    public required BitmapSource Bitmap { get; init; }  
    public required ConcurrentDictionary<int, CroppedBitmap> CropCache { get; init; }  
}
```

```
// -----
```

```
// Viewport-only renderer
```

```
// -----
```

```
private Image? _viewportImage;
```

```
private readonly DispatcherTimer _renderDebounce;
```

```
public MainWindow()
```

```

{
    InitializeComponent();

    _projectRoot = ResolveProjectRoot();

    HookFullscreenGuards();

    Loaded += (_, _) =>
    {
        ForceFullscreen();

        Boot();

        Dispatcher.BeginInvoke(DispatcherPriority.Input, new
Action(FocusGameSurface));
    };

    _renderDebounce = new DispatcherTimer { Interval =
TimeSpan.FromMilliseconds(16) };

    _renderDebounce.Tick += (_, _) =>
    {
        _renderDebounce.Stop();

        RenderViewport();
    };
}

private void FocusGameSurface()
{
    Keyboard.Focus(ViewCanvas);

    ViewCanvas.Focus();
}

```

```

// -----
// Project root resolution
// -----

private static string ResolveProjectRoot()
{
    var start = AppContext.BaseDirectory;

    var candidates = new[]
    {
        start,
        Directory.GetCurrentDirectory()
    }
    .Where(p => !string.IsNullOrEmpty(p))
    .Distinct(StringComparer.OrdinalIgnoreCase)
    .ToArray();

    const int maxUp = 8;

    foreach (var baseDir in candidates)
    {
        try
        {
            {
                var dir = new DirectoryInfo(baseDir);
                for (int i = 0; i < maxUp && dir is not null; i++)
                {
                    var probe = IOPath.Combine(dir.FullName, "Vitriol.Projects");
                    if (Directory.Exists(probe))

```

```

        return probe;

        dir = dir.Parent;
    }
}
catch {}
}

return IOPath.Combine(Directory.GetCurrentDirectory(), "Vitriol.Projects");
}

private void Boot()
{
    HudText.Text = $"Runtime ready. Click 'Load Map'. (Project: {_projectRoot})";
}

// -----
// UI: Load Map button
// -----

private void OnLoadMapClick(object sender, RoutedEventArgs e)
{
    try
    {
        if (!Directory.Exists(MapsFolder))
        {
            HudText.Text = $"Maps folder not found: {MapsFolder}";
            FocusGameSurface();
            return;
        }
    }
}

```

```

    }

    var dlg = new OpenFileDialog
    {
        Title = "Open Vitriol Map",
        Filter = "Vitriol Map (*.vitmap.json)|*.vitmap.json|JSON (*.json)|*.json|All files (*.*)|**",
        InitialDirectory = MapsFolder,
        CheckFileExists = true,
        Multiselect = false
    };

    if (dlg.ShowDialog(this) != true)
    {
        FocusGameSurface();
        return;
    }

    _playerX = 0;
    _playerY = 0;

    LoadMap(dlg.FileName, "Manual Load");

    Dispatcher.BeginInvoke(DispatcherPriority.Input, new
    Action(FocusGameSurface));
}

catch (Exception ex)
{
    HudText.Text = $"Load map dialog failed: {ex.Message}";

    FocusGameSurface();
}

```

```

    }
}

// -----
// Map loading
// -----

private void LoadMap(string path, string reason)
{
    try
    {
        _mapPath = path;
        _map = MapSerializer.LoadFromFile(path);

        if (_map.Events is null)
            _map = _map with { Events = Array.Empty<MapEventDto>() };

        EnsureCollisionLayer();

        var errors = ValidateTilesetFilesExist(_map);
        if (errors.Count > 0)
        {
            _map = null;
            HudText.Text = "Map load blocked:\n" + string.Join("\n", errors);
            return;
        }

        MarkTilesetsDirty();
        EnsureTilesetsCompiled();
    }
}

```

```
_playerX = Math.Clamp(_playerX, 0, _map.Width - 1);
```

```
_playerY = Math.Clamp(_playerY, 0, _map.Height - 1);
```

```
EnsureViewportImageExists();
```

```
ViewCanvas.Width = checked(_map.Width * _map.TileSize);
```

```
ViewCanvas.Height = checked(_map.Height * _map.TileSize);
```

```
    HudText.Text = $"{reason}: Loaded {IOPath.GetFileName(path)}  
    Player({_playerX},{_playerY})";
```

```
    Dispatcher.BeginInvoke(DispatcherPriority.Loaded, new Action(() =>  
    {  
        CenterCameraOnPlayer();  
        RequestRender();  
        FocusGameSurface();  
    }));  
}  
catch (Exception ex)  
{  
    HudText.Text = $"Load failed: {ex.Message}";  
    FocusGameSurface();  
}  
}
```

```
private List<string> ValidateTilesetFilesExist(MapDto map)  
{
```

```

var issues = new List<string>();

var tilesets = map.Tilesets ?? Array.Empty<TilesetRefDto>();
foreach (var ts in tilesets)
{
    var img = (ts.ImagePath ?? "").Trim();

    if (img.Length == 0)
        continue;

    var full = ResolveTilesetPathFromImagePath(img);
    if (full is null)
    {
        issues.Add($"Tileset '{ts.TilesetId}': ImagePath invalid: '{img}'.");
        continue;
    }

    if (!File.Exists(full))
        issues.Add($"Tileset '{ts.TilesetId}': missing PNG: '{full}'.");
}

return issues;
}

private void EnsureCollisionLayer()
{
    if (_map is null) return;

```



```

bool hasCollision =
    _map.Layers.Any(l => l.Kind == LayerKind.Collision || l.LayerId == "col");

if (hasCollision) return;

var col = new TileLayerDto(
    LayerId: "col",
    Name: "Collision",
    Kind: LayerKind.Collision,
    Visible: true,
    Tiles: new int[_map.Width * _map.Height]
);

_map = _map with { Layers = _map.Layers.Concat(new[] { col }).ToArray() };
}

private string? ResolveTilesetPathFromImagePath(string imagePath)
{
    if (string.IsNullOrEmpty(imagePath))
        return null;

    imagePath = imagePath.Trim();

    if (IOPath.IsPathRooted(imagePath))
        return imagePath;

    var rel = imagePath.Replace('/', '\\');
    return IOPath.Combine(_projectRoot, rel);
}

```

```
}
```

```
private TileLayerDto? GetBackgroundLayer() =>
```

```
    _map?.Layers.FirstOrDefault(l => l.Kind == LayerKind.Background);
```

```
private TileLayerDto? GetCollisionLayer() =>
```

```
    _map?.Layers.FirstOrDefault(l => l.Kind == LayerKind.Collision || l.LayerId == "col");
```

```
private string? ResolveTilesetFullPath(TilesetRefDto ts)
```

```
{
```

```
    var raw = (ts.ImagePath ?? "").Trim();
```

```
    if (raw.Length > 0 && IOPath.IsPathRooted(raw) && File.Exists(raw))
```

```
        return raw;
```

```
    if (raw.Length > 0)
```

```
    {
```

```
        var fileName = IOPath.GetFileName(raw.Replace('/', '\\'));
        if (!string.IsNullOrEmpty(fileName))
```

```
        {
```

```
            {
```

```
                var inTilesets = IOPath.Combine(PngsFolder, fileName);
```

```
                if (File.Exists(inTilesets))
```

```
                    return inTilesets;
```

```
            if (IOPath.GetExtension(fileName).Length == 0)
```

```
            {
```

```
                var inTilesetsPng = IOPath.Combine(PngsFolder, fileName + ".png");
```

```
                if (File.Exists(inTilesetsPng))
```

```
                    return inTilesetsPng;
```

```
    }  
  }  
}
```

```
var fallbackName = (ts.TilesetId ?? "").Trim();  
if (fallbackName.Length > 0)  
{  
    var p = IOPath.Combine(PngsFolder, fallbackName + ".png");  
    if (File.Exists(p))  
        return p;  
}
```

```
    return null;  
}
```

```
private void MarkTilesetsDirty()  
{  
    lock (_tilesetCompileLock)  
    {  
        _tilesetsDirty = true;  
    }  
}
```

```
private void EnsureTilesetsCompiled()  
{  
    var map = _map;  
  
    lock (_tilesetCompileLock)
```

```

{
    if (!_tilesetsDirty) return;

    if (map is null)
    {
        _compiledTilesets = new List<TilesetRuntime>();
        _tilesetsDirty = false;
        return;
    }

    var compiled = new List<TilesetRuntime>();
    int runningStart = 0;

    var tilesets = map.Tilesets ?? Array.Empty<TilesetRefDto>();
    foreach (var ts in tilesets)
    {
        var img = (ts.ImagePath ?? "").Trim();

        if (img.Length == 0)
            continue;

        var fullPath = ResolveTilesetPathFromImagePath(img);
        if (fullPath is null || !File.Exists(fullPath))
            continue;

        var bmp = _tilesetBitmapCache.GetOrAdd(fullPath, LoadBitmap);
        var meta = ComputeMeta(ts, bmp);
    }
}

```

```
var cropCache = _tileCropCache.GetOrAdd(fullPath, _ => new  
ConcurrentDictionary<int, CroppedBitmap>());
```

```
compiled.Add(new TilesetRuntime  
{  
    Ref = ts,  
    FullPath = fullPath,  
    StartIndexGlobal = runningStart,  
    TileCount = meta.TileCount,  
    TilesAcross = meta.TilesAcross,  
    TileW = meta.TileWidth,  
    TileH = meta.TileHeight,  
    Margin = meta.Margin,  
    Spacing = meta.Spacing,  
    Bitmap = bmp,  
    CropCache = cropCache  
});
```

```
runningStart += meta.TileCount;  
}
```

```
_compiledTilesets = compiled;  
_tilesetsDirty = false;  
}  
}
```

```
private static TilesetMeta ComputeMeta(TilesetRefDto ts, BitmapSource bmp)  
{
```

```
int tileW = Math.Max(1, ts.TileWidth);
int tileH = Math.Max(1, ts.TileHeight);
int margin = Math.Max(0, ts.Margin);
int spacing = Math.Max(0, ts.Spacing);

int stepX = tileW + spacing;
int stepY = tileH + spacing;

int tilesAcross = stepX > 0 ? (bmp.PixelWidth - margin) / stepX : 0;
int tilesDown = stepY > 0 ? (bmp.PixelHeight - margin) / stepY : 0;

tilesAcross = Math.Max(0, tilesAcross);
tilesDown = Math.Max(0, tilesDown);

int tileCount = tilesAcross * tilesDown;

return new TilesetMeta(
    PixelWidth: bmp.PixelWidth,
    PixelHeight: bmp.PixelHeight,
    TileWidth: tileW,
    TileHeight: tileH,
    Margin: margin,
    Spacing: spacing,
    TilesAcross: tilesAcross,
    TilesDown: tilesDown,
    TileCount: tileCount
);
}
```

```

private TilesetRuntime? FindRuntimeForGlobalTile(int globalTileIndex)
{
    EnsureTilesetsCompiled();

    if (globalTileIndex < 0) return null;

    foreach (var ts in _compiledTilesets)
    {
        if (globalTileIndex >= ts.StartIndexGlobal &&
            globalTileIndex < ts.StartIndexGlobal + ts.TileCount)
            return ts;
    }

    return null;
}

```

```

private static int GetLocalIndex(TilesetRuntime ts, int globalTileIndex) =>
    globalTileIndex - ts.StartIndexGlobal;

```

```

// -----

```

```

// Camera

```

```

// -----

```

```

private void CenterCameraOnPlayer()

```

```

{

```

```

    var map = _map;

```

```

    if (map is null) return;

```

```

    if (ViewScroll.ViewportWidth <= 0 || ViewScroll.ViewportHeight <= 0)

```

```

{
    Dispatcher.BeginInvoke(DispatcherPriority.Loaded, new
Action(CenterCameraOnPlayer));

    return;
}

double mapPxW = map.Width * map.TileSize;
double mapPxH = map.Height * map.TileSize;

double playerCenterX = _playerX * map.TileSize + map.TileSize / 2.0;
double playerCenterY = _playerY * map.TileSize + map.TileSize / 2.0;

double desiredX = playerCenterX - ViewScroll.ViewportWidth / 2.0;
double desiredY = playerCenterY - ViewScroll.ViewportHeight / 2.0;

double maxX = Math.Max(0, mapPxW - ViewScroll.ViewportWidth);
double maxY = Math.Max(0, mapPxH - ViewScroll.ViewportHeight);

desiredX = Math.Clamp(desiredX, 0, maxX);
desiredY = Math.Clamp(desiredY, 0, maxY);

ViewScroll.ScrollToHorizontalOffset(desiredX);
ViewScroll.ScrollToVerticalOffset(desiredY);
}

private void DeadzoneFollowPlayer()
{
    var map = _map;

```



```
if (map is null) return;
```

```
if (ViewScroll.ViewportWidth <= 0 || ViewScroll.ViewportHeight <= 0) return;
```

```
double mapPxW = map.Width * map.TileSize;
```

```
double mapPxH = map.Height * map.TileSize;
```

```
double camX = ViewScroll.HorizontalOffset;
```

```
double camY = ViewScroll.VerticalOffset;
```

```
double vw = ViewScroll.ViewportWidth;
```

```
double vh = ViewScroll.ViewportHeight;
```

```
double dzW = vw * DeadzoneFracX;
```

```
double dzH = vh * DeadzoneFracY;
```

```
double dzLeft = camX + (vw - dzW) / 2.0;
```

```
double dzTop = camY + (vh - dzH) / 2.0;
```

```
double dzRight = dzLeft + dzW;
```

```
double dzBottom = dzTop + dzH;
```

```
double px = _playerX * map.TileSize + map.TileSize / 2.0;
```

```
double py = _playerY * map.TileSize + map.TileSize / 2.0;
```

```
if (px < dzLeft) camX -= (dzLeft - px);
```

```
else if (px > dzRight) camX += (px - dzRight);
```

```
if (py < dzTop) camY -= (dzTop - py);
```

```
else if (py > dzBottom) camY += (py - dzBottom);
```

```

double maxX = Math.Max(0, mapPxW - vw);
double maxY = Math.Max(0, mapPxH - vh);

camX = Math.Clamp(camX, 0, maxX);
camY = Math.Clamp(camY, 0, maxY);

ViewScroll.ScrollToHorizontalOffset(camX);
ViewScroll.ScrollToVerticalOffset(camY);
}

// -----
// Rendering
// -----

private void EnsureViewportImageExists()
{
    if (_viewportImage != null) return;

    _viewportImage = new Image
    {
        Stretch = Stretch.None,
        SnapsToDevicePixels = true
    };

    RenderOptions.SetBitmapScalingMode(_viewportImage,
BitmapScalingMode.NearestNeighbor);

ViewCanvas.Children.Clear();
ViewCanvas.Children.Add(_viewportImage);

```

```
}
```

```
private void RequestRender()
```

```
{
```

```
    if (_map is null) return;
```

```
    _renderDebounce.Stop();
```

```
    _renderDebounce.Start();
```

```
}
```

```
private void OnScrollChanged(object sender, ScrollChangedEventArgs e)
```

```
{
```

```
    if (_map is null) return;
```

```
    RequestRender();
```

```
}
```

```
private void RenderViewport()
```

```
{
```

```
    var map = _map;
```

```
    if (map is null) return;
```

```
    EnsureViewportImageExists();
```

```
    EnsureTilesetsCompiled();
```

```
    var bg = GetBackgroundLayer();
```

```
    if (bg is null) { HudText.Text = "Map has no background layer."; return; }
```

```
    int mapPxW = checked(map.Width * map.TileSize);
```

```
    int mapPxH = checked(map.Height * map.TileSize);
```

```
double viewLeft = ViewScroll.HorizontalOffset;
double viewTop = ViewScroll.VerticalOffset;
double viewW = ViewScroll.ViewportWidth;
double viewH = ViewScroll.ViewportHeight;

int pad = map.TileSize * 2;
int regionX = ClampToInt(viewLeft) - pad;
int regionY = ClampToInt(viewTop) - pad;
int regionW = ClampToInt(viewW) + pad * 2;
int regionH = ClampToInt(viewH) + pad * 2;

if (regionX < 0) regionX = 0;
if (regionY < 0) regionY = 0;
if (regionX + regionW > mapPxW) regionW = mapPxW - regionX;
if (regionY + regionH > mapPxH) regionH = mapPxH - regionY;

if (regionW <= 1 || regionH <= 1) return;

int startTileX = regionX / map.TileSize;
int startTileY = regionY / map.TileSize;
int endTileX = (regionX + regionW + map.TileSize - 1) / map.TileSize;
int endTileY = (regionY + regionH + map.TileSize - 1) / map.TileSize;

startTileX = Math.Max(0, startTileX);
startTileY = Math.Max(0, startTileY);
endTileX = Math.Min(map.Width, endTileX);
endTileY = Math.Min(map.Height, endTileY);
```

```

var visual = new DrawingVisual();
using (var dc = visual.RenderOpen())
{
    dc.DrawRectangle(Brushes.Black, null, new Rect(0, 0, regionW, regionH));

    for (int y = startTileY; y < endTileY; y++)
    for (int x = startTileX; x < endTileX; x++)
    {
        int idx = y * map.Width + x;
        if ((uint)idx >= (uint)bg.Tiles.Length) continue;

        int t = bg.Tiles[idx];
        if (t < 0) continue;

        var ts = FindRuntimeForGlobalTile(t);
        if (ts is null || ts.TileCount <= 0) continue;

        int local = GetLocalIndex(ts, t);
        if ((uint)local >= (uint)ts.TileCount) continue;

        var crop = ts.CropCache.GetOrAdd(local, tileIndex =>
        {
            int cx = ts.TilesAcross > 0 ? (tileIndex % ts.TilesAcross) : 0;
            int cy = ts.TilesAcross > 0 ? (tileIndex / ts.TilesAcross) : 0;

            int px = ts.Margin + cx * (ts.TileW + ts.Spacing);
            int py = ts.Margin + cy * (ts.TileH + ts.Spacing);

```

```

        if (px + ts.TileW > ts.Bitmap.PixelWidth || py + ts.TileH > ts.Bitmap.PixelHeight)
        {
            var dummy = new CroppedBitmap(ts.Bitmap, new Int32Rect(0, 0, 1, 1));
            dummy.Freeze();
            return dummy;
        }

        var cb = new CroppedBitmap(ts.Bitmap, new Int32Rect(px, py, ts.TileW,
ts.TileH));
        cb.Freeze();
        return cb;
    });

    double drawX = x * map.TileSize - regionX;
    double drawY = y * map.TileSize - regionY;

    dc.DrawImage(crop, new Rect(drawX, drawY, map.TileSize, map.TileSize));
}

if (map.Events is not null && map.Events.Count > 0)
{
    var triggerFill = new SolidColorBrush(Color.FromArgb(220, 255, 215, 0));
    triggerFill.Freeze();

    var npcFill = new SolidColorBrush(Color.FromArgb(220, 80, 170, 255));
    npcFill.Freeze();

    var warpFill = new SolidColorBrush(Color.FromArgb(220, 190, 90, 255));
    warpFill.Freeze();

    var outline = new Pen(Brushes.Black, 1); outline.Freeze();

```

```

foreach (var ev in map.Events)
{
    if (ev.X < startTileX || ev.X >= endTileX || ev.Y < startTileY || ev.Y >= endTileY)
        continue;

    double left = ev.X * map.TileSize - regionX;
    double top = ev.Y * map.TileSize - regionY;
    var type = (ev.Type ?? "").Trim();

    if (type.Equals("NPC", StringComparison.OrdinalIgnoreCase))
    {
        var c = new Point(left + map.TileSize / 2.0, top + map.TileSize / 2.0);
        dc.DrawEllipse(npcFill, outline, c, map.TileSize * 0.22, map.TileSize * 0.22);
    }
    else if (type.Equals("Warp", StringComparison.OrdinalIgnoreCase))
    {
        var cx = left + map.TileSize / 2.0;
        var cy = top + map.TileSize / 2.0;
        double r = map.TileSize * 0.24;

        var geo = new StreamGeometry();
        using (var g = geo.Open())
        {
            g.BeginFigure(new Point(cx, cy - r), true, true);
            g.LineTo(new Point(cx + r, cy), true, false);
            g.LineTo(new Point(cx, cy + r), true, false);
            g.LineTo(new Point(cx - r, cy), true, false);
        }
    }
}

```

```

    }
    geo.Freeze();
    dc.DrawGeometry(warpFill, outline, geo);
}
else
{
    dc.DrawRectangle(triggerFill, outline,
        new Rect(left + map.TileSize * 0.28, top + map.TileSize * 0.28,
            map.TileSize * 0.44, map.TileSize * 0.44));
}
}
}

```

```

if (_playerX >= startTileX && _playerX < endTileX && _playerY >= startTileY &&
_playerY < endTileY)
{
    var fill = new SolidColorBrush(Color.FromArgb(230, 90, 180, 255)); fill.Freeze();
    var outline = new Pen(Brushes.Black, 1); outline.Freeze();

    double px = _playerX * map.TileSize - regionX;
    double py = _playerY * map.TileSize - regionY;

    dc.DrawEllipse(fill, outline,
        new Point(px + map.TileSize / 2.0, py + map.TileSize / 2.0),
        map.TileSize * 0.28,
        map.TileSize * 0.28);
}
}

```



```
var rtb = new RenderTargetBitmap(regionW, regionH, 96, 96, PixelFormats.Pbgra32);
rtb.Render(visual);
rtb.Freeze();

_viewportImage!.Source = rtb;
_viewportImage.Width = regionW;
_viewportImage.Height = regionH;

Canvas.SetLeft(_viewportImage, regionX);
Canvas.SetTop(_viewportImage, regionY);
}
```

```
private static int ClampToInt(double v)
{
    if (v <= int.MinValue) return int.MinValue;
    if (v >= int.MaxValue) return int.MaxValue;
    return (int)Math.Floor(v);
}
```

```
private static BitmapSource LoadBitmap(string path)
{
    var bmp = new BitmapImage();
    bmp.BeginInit();
    bmp.CacheOption = BitmapCacheOption.OnLoad;
    bmp.UriSource = new Uri(path, UriKind.Absolute);
    bmp.EndInit();
    bmp.Freeze();
}
```

```

        return bmp;
    }

    // -----
    // Input + gameplay
    // -----

    private void OnWindowKeyDown(object sender, KeyEventArgs e)
    {
        if (_map is null) return;

        if (e.Key == Key.Escape)
        {
            Close();
            return;
        }

        if (e.Key == Key.E || e.Key == Key.Enter || e.Key == Key.Space)
        {
            InteractAtPlayer();
            e.Handled = true;
            return;
        }

        switch (e.Key)
        {
            case Key.Up:
            case Key.W: TryMove(0, -1); e.Handled = true; return;
            case Key.Down:

```

```

        case Key.S: TryMove(0, 1); e.Handled = true; return;
        case Key.Left:
        case Key.A: TryMove(-1, 0); e.Handled = true; return;
        case Key.Right:
        case Key.D: TryMove(1, 0); e.Handled = true; return;
    }
}

```

```

private void TryMove(int dx, int dy)
{
    var map = _map;
    if (map is null) return;

    int nx = _playerX + dx;
    int ny = _playerY + dy;

    if (nx < 0 || ny < 0 || nx >= map.Width || ny >= map.Height)
    {
        HudText.Text = "Blocked: edge of map.";
        return;
    }

    var col = GetCollisionLayer();
    if (col is not null && col.Tiles.Length == map.Width * map.Height)
    {
        int idx = ny * map.Width + nx;
        if (idx >= 0 && idx < col.Tiles.Length && col.Tiles[idx] != 0)
        {

```

```

        HudText.Text = "Blocked: collision tile.";
        return;
    }
}

_playerX = nx;
_playerY = ny;

var ev = GetEventAt(_playerX, _playerY);
if (ev is not null)
{
    if (string.Equals(ev.Type, "Warp", StringComparison.OrdinalIgnoreCase))
        DoWarp(ev);
    else if (string.Equals(ev.Type, "Trigger", StringComparison.OrdinalIgnoreCase))
        ExecuteEventScript(ev);
}

HudText.Text = $"Player({_playerX},{_playerY})";

DeadzoneFollowPlayer();
RequestRender();
}

private void InteractAtPlayer()
{
    var ev = GetEventAt(_playerX, _playerY);
    if (ev is null)
    {

```

```
    HudText.Text = "Interact: no event here.";
    return;
}
```

```
if (string.Equals(ev.Type, "NPC", StringComparison.OrdinalIgnoreCase) ||
    string.Equals(ev.Type, "Trigger", StringComparison.OrdinalIgnoreCase))
{
    ExecuteEventScript(ev);
    return;
}
```

```
    HudText.Text = $"Interact: {ev.Name} ({ev.Type})";
}
```

```
private MapEventDto? GetEventAt(int x, int y)
{
    var map = _map;
    if (map?.Events is null || map.Events.Count == 0) return null;
    return map.Events.FirstOrDefault(e => e.X == x && e.Y == y);
}
```

```
private void ExecuteEventScript(MapEventDto ev)
{
    var cmds = ev.Commands ?? Array.Empty<EventCommandDto>();
    if (cmds.Count == 0)
    {
        HudText.Text = $"Event '{ev.Name}' has no commands.";
        return;
    }
}
```

```
}
```

```
foreach (var cmd in cmds)
```

```
{
```

```
    switch ((cmd.Command ?? "").Trim())
```

```
    {
```

```
        case "ShowText":
```

```
        {
```

```
            cmd.Args.TryGetValue("text", out var text);
```

```
            text ??= "";
```

```
            if (text.Length == 0) text = "(empty text)";
```

```
            MessageBox.Show(this, text, ev.Name);
```

```
            break;
```

```
        }
```

```
        case "SetFlag":
```

```
        {
```

```
            cmd.Args.TryGetValue("flag", out var flag);
```

```
            cmd.Args.TryGetValue("value", out var value);
```

```
            flag = (flag ?? "").Trim();
```

```
            value ??= "";
```

```
            if (flag.Length == 0)
```

```
            {
```

```
                HudText.Text = "SetFlag failed: flag is empty.";
```

```
                break;
```

```
            }
```

```

        _flags[flag] = value;

        HudText.Text = $"Flag set: {flag}='{value}'";

        break;
    }

    default:

        HudText.Text = $"Unknown command: {cmd.Command}";

        break;
    }
}

```

```

private void DoWarp(MapEventDto ev)
{
    var map = _map;

    if (map is null) return;

    var p = ev.Parameters ?? new Dictionary<string, string>();

    p.TryGetValue("destMap", out var destMap);
    p.TryGetValue("destX", out var destXStr);
    p.TryGetValue("destY", out var destYStr);

    destMap = (destMap ?? "").Trim();
    if (string.IsNullOrEmpty(destMap))
    {
        HudText.Text = "Warp failed: destMap missing.";

        return;
    }
}

```

```
}
```

```
int.TryParse(destXStr, out var destX);
```

```
int.TryParse(destYStr, out var destY);
```

```
var destPath = IOPath.Combine(MapsFolder, destMap +  
MapFormat.DefaultExtension);
```

```
if (!File.Exists(destPath))
```

```
{
```

```
    HudText.Text = $"Warp failed: map not found '{destMap}'";
```

```
    return;
```

```
}
```

```
LoadMap(destPath, $"Warp to {destMap}");
```

```
if (_map is null) return;
```

```
_playerX = Math.Clamp(destX, 0, _map.Width - 1);
```

```
_playerY = Math.Clamp(destY, 0, _map.Height - 1);
```

```
HudText.Text = $"Warped: {ev.Name} -> {destMap} ({_playerX},{_playerY})";
```

```
Dispatcher.BeginInvoke(DispatcherPriority.Background, new Action(() =>
```

```
{
```

```
    CenterCameraOnPlayer();
```

```
    RequestRender();
```

```
    FocusGameSurface();
```

```
});
```



```

    }
}

Vitriol.Runtime\Vitriol.Runtime.csproj
<Project Sdk="Microsoft.NET.Sdk">

  <ItemGroup>

    <ProjectReference Include="..\Vitriol.Engine\Vitriol.Engine.csproj" />

    <ProjectReference Include="..\Vitriol.Shared\Vitriol.Shared.csproj" />

  </ItemGroup>

  <PropertyGroup>

    <OutputType>WinExe</OutputType>

    <TargetFramework>net9.0-windows</TargetFramework>

    <Nullable>enable</Nullable>

    <ImplicitUsings>enable</ImplicitUsings>

    <UseWPF>true</UseWPF>

  </PropertyGroup>

</Project>

```

Vitriol.Shared

Vitriol.Shared\Ids

Vitriol.Shared\Ids\EntityId.cs

```
namespace Vitriol.Shared.Ids;
```

```

public readonly record struct EntityId(long Value)
{
    public override string ToString() => Value.ToString();
}

```

[Vitriol.Shared\Maps](#)

[Vitriol.Shared\Maps\EventCommandDto.cs](#)

```
namespace Vitriol.Shared.Maps;
```

```
public sealed record EventCommandDto(  
    string Command,  
    Dictionary<string, string> Args  
);
```

[Vitriol.Shared\Maps\MapDto.cs](#)

```
using System.Collections.Generic;
```

```
namespace Vitriol.Shared.Maps;
```

```
public sealed record MapDto(  
    int FormatVersion,  
    string MapId,  
    string Name,  
    int Width,  
    int Height,  
    int TileSize,  
    IReadOnlyList<TilesetRefDto> Tilesets,  
    IReadOnlyList<TileLayerDto> Layers  
)  
{  
    public IReadOnlyList<MapEventDto> Events { get; init; }  
        = System.Array.Empty<MapEventDto>();  
}
```

[Vitriol.Shared\Maps\MapEventDto.cs](#)

```
namespace Vitriol.Shared.Maps;
```

```
public sealed record MapEventDto(
    string EventId,
    string Name,
    string Type,
    int X,
    int Y,
    Dictionary<string, string>? Parameters = null,
    IReadOnlyList<EventCommandDto>? Commands = null
);
```

[Vitriol.Shared\Maps\TileLayerDto.cs](#)

```
namespace Vitriol.Shared.Maps;
```

```
public sealed record TileLayerDto(
    string LayerId,
    string Name,
    LayerKind Kind,
    bool Visible,
    int[] Tiles // length = Width*Height, tile index (or -1 for empty)
);
```

```
public enum LayerKind
```

```
{
    Background = 0,
    Collision = 1,
    Objects = 2,
    Overlay = 3
}
```

Vitriol.Shared\Maps\TilesetRefDto.cs

```
namespace Vitriol.Shared.Maps;
```

```
public sealed record TilesetRefDto(
```

```
    string TilesetId,
```

```
    string ImagePath,    // relative to project root, e.g. "assets/tilesets/kanto.png"
```

```
    int TileWidth,
```

```
    int TileHeight,
```

```
    int Margin,
```

```
    int Spacing
```

```
);
```

Vitriol.Shared\Pokemon

Vitriol.Shared\Pokemon\PokemonSpeciesDto.cs

```
namespace Vitriol.Shared.Pokemon;
```

```
public sealed record PokemonSpeciesDto(
```

```
    long SpeciesId,
```

```
    string Name,
```

```
    string PrimaryType,
```

```
    string? SecondaryType,
```

```
    int BaseHP,
```

```
    int BaseAttack,
```

```
    int BaseDefense,
```

```
    int BaseSpAttack,
```

```
    int BaseSpDefense,
```

```
    int BaseSpeed
```

```
);
```

Vitriol.Shared\Validation

Vitriol.Shared\Validation\MapValidator.cs

```
using Vitriol.Shared.Maps;
```

```
namespace Vitriol.Shared.Validation;
```

```
public static class MapValidator
```

```
{
```

```
    public static IReadOnlyList<ValidationError> Validate(MapDto map)
```

```
    {
```

```
        var errors = new List<ValidationError>();
```

```
        if (map.FormatVersion <= 0)
```

```
            errors.Add(new("FormatVersion", "FormatVersion must be a positive integer."));
```

```
        if (string.IsNullOrEmpty(map.MapId))
```

```
            errors.Add(new("MapId", "MapId is required."));
```

```
        if (map.Width <= 0 || map.Height <= 0)
```

```
            errors.Add(new("Size", "Width and Height must be positive."));
```

```
        if (map.TileSize <= 0)
```

```
            errors.Add(new("TileSize", "TileSize must be positive."));
```

```
        foreach (var layer in map.Layers)
```

```
        {
```

```
            if (layer.Tiles is null)
```

```
            {
```

```

        errors.Add(new($"Layer:{layer.LayerId}", "Tiles array is missing."));
        continue;
    }

    var expected = map.Width * map.Height;

    if (layer.Tiles.Length != expected)

        errors.Add(new($"Layer:{layer.LayerId}", $"Tiles length must be Width*Height
({expected})."));

    }

    return errors;
}
}

```

[Vitriol.Shared.Validation.SpeciesValidator.cs](#)

```
using Vitriol.Shared.Pokemon;
```

```
namespace Vitriol.Shared.Validation;
```

```
public static class SpeciesValidator
```

```

{
    public static IReadOnlyList<ValidationError> Validate(PokemonSpeciesDto dto)
    {
        var errors = new List<ValidationError>();

        if (string.IsNullOrEmpty(dto.Name))
            errors.Add(new("Name", "Name is required."));

        if (string.IsNullOrEmpty(dto.PrimaryType))
            errors.Add(new("PrimaryType", "Primary type is required."));
    }
}

```

```

        if (!string.IsNullOrEmpty(dto.SecondaryType) &&
            string.Equals(dto.PrimaryType, dto.SecondaryType,
                StringComparison.OrdinalIgnoreCase))
            errors.Add(new("SecondaryType", "Secondary type cannot be the same as primary
                type."));

        int[] stats = [dto.BaseHP, dto.BaseAttack, dto.BaseDefense, dto.BaseSpAttack,
            dto.BaseSpDefense, dto.BaseSpeed];

        if (stats.Any(s => s < 1 || s > 255))
            errors.Add(new("BaseStats", "Each base stat must be between 1 and 255."));

        return errors;
    }
}

```

Vitriol.Shared\Validation\ValidationError.cs

```
namespace Vitriol.Shared.Validation;
```

```
public sealed record ValidationError(string Field, string Message);
```

Vitriol.Shared\Class1.cs

```
namespace Vitriol.Shared;
```

```
public class Class1
```

```
{
```

```
}
```

Vitriol.Shared\Vitriol.Shared.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
<PropertyGroup>
```

```
<TargetFramework>net9.0</TargetFramework>

<ImplicitUsings>enable</ImplicitUsings>

<Nullable>enable</Nullable>

</PropertyGroup>

</Project>
```

Vitriol.Tests

(do not mark this as part of the testing section)

Vitriol.Tests\DatabaseSmokeTests.cs

```
using Microsoft.Data.Sqlite;
```

```
using NUnit.Framework;
```

```
using Vitriol.Data.Migrations;
```

```
using Vitriol.Data.Repositories;
```

```
using Vitriol.Shared.Pokemon;
```

```
using Vitriol.Shared.Validation;
```

```
namespace Vitriol.Tests;
```

```
public class DatabaseSmokeTests
```

```
{
```

```
    [Test]
```

```
    public void Migration_creates_species_table_and_allows_insert_and_query()
```

```
    {
```

```
        using var conn = new SqliteConnection("Data Source=:memory:");
```

```
        conn.Open();
```

```
        new MigrationRunner().EnsureMigrated(conn);
```



```
var repo = new PokemonSpeciesRepository();
var bulbasaur = new PokemonSpeciesDto(
    SpeciesId: 0,
    Name: "Bulbasaur",
    PrimaryType: "Grass",
    SecondaryType: "Poison",
    BaseHP: 45,
    BaseAttack: 49,
    BaseDefense: 49,
    BaseSpAttack: 65,
    BaseSpDefense: 65,
    BaseSpeed: 45
);

var errors = SpeciesValidator.Validate(bulbasaur);
Assert.That(errors, Is.Empty);

var id = repo.Insert(conn, bulbasaur);
Assert.That(id, Is.GreaterThan(0));

var loaded = repo.GetByName(conn, "Bulbasaur");
Assert.That(loaded, Is.Not.Null);
Assert.That(loaded!.PrimaryType, Is.EqualTo("Grass"));
Assert.That(loaded.SecondaryType, Is.EqualTo("Poison"));
}
```

[Test]

```

public void Validator_rejects_duplicate_types()
{
    var dto = new PokemonSpeciesDto(
        SpeciesId: 0,
        Name: "Testmon",
        PrimaryType: "Fire",
        SecondaryType: "Fire",
        BaseHP: 50, BaseAttack: 50, BaseDefense: 50, BaseSpAttack: 50,
        BaseSpDefense: 50, BaseSpeed: 50
    );

    var errors = SpeciesValidator.Validate(dto);
    Assert.That(errors.Any(e => e.Field == "SecondaryType"), Is.True);
}
}

```

Vitriol.Tests\MapFormatTests.cs

```

using NUnit.Framework;
using Vitriol.Engine.Maps;
using Vitriol.Shared.Maps;
using Vitriol.Shared.Validation;

```

```

namespace Vitriol.Tests;

```

```

public class MapFormatTests
{
    [Test]
    public void Map_json_round_trip_preserves_dimensions_and_tiles()
    {
        var w = 8;
    }
}

```

```
var h = 6;
```

```
var tiles = new int[w * h];
```

```
for (int i = 0; i < tiles.Length; i++) tiles[i] = i % 3;
```

```
var map = new MapDto(
```

```
    FormatVersion: MapFormat.CurrentVersion,
```

```
    MapId: "round_trip",
```

```
    Name: "Round Trip",
```

```
    Width: w,
```

```
    Height: h,
```

```
    TileSize: 32,
```

```
    Tilesets: new List<TilesetRefDto>(),
```

```
    Layers: new List<TileLayerDto>
```

```
    {
```

```
        new("bg", "Background", LayerKind.Background, true, tiles)
```

```
    }
```

```
);
```

```
var errors = MapValidator.Validate(map);
```

```
Assert.That(errors, Is.Empty);
```

```
var json = MapSerializer.ToJson(map);
```

```
var loaded = MapSerializer.FromJson(json);
```

```
Assert.That(loaded.Width, Is.EqualTo(map.Width));
```

```
Assert.That(loaded.Height, Is.EqualTo(map.Height));
```

```
Assert.That(loaded.Layers.Count, Is.EqualTo(1));
```

```
        Assert.That(loaded.Layers[0].Tiles, Is.EqualTo(map.Layers[0].Tiles));
    }
}
```

Vitriol.Tests\UnitTest1.csj

```
namespace Vitriol.Tests;
```

```
public class Tests
```

```
{
```

```
    [SetUp]
```

```
    public void Setup()
```

```
    {
```

```
    }
```

```
    [Test]
```

```
    public void Test1()
```

```
    {
```

```
        Assert.Pass();
```

```
    }
```

```
}
```

Vitriol.Tests\Vitriol.Tests.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
  <PropertyGroup>
```

```
    <TargetFramework>net9.0</TargetFramework>
```

```
    <LangVersion>latest</LangVersion>
```

```
    <ImplicitUsings>enable</ImplicitUsings>
```

```
    <Nullable>enable</Nullable>
```

```
    <IsPackable>>false</IsPackable>
```

</PropertyGroup>

<ItemGroup>

<PackageReference Include="coverlet.collector" Version="6.0.2" />

<PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />

<PackageReference Include="NUnit" Version="4.2.2" />

<PackageReference Include="NUnit.Analyzers" Version="4.4.0" />

<PackageReference Include="NUnit3TestAdapter" Version="4.6.0" />

</ItemGroup>

<ItemGroup>

<Using Include="NUnit.Framework" />

</ItemGroup>

<ItemGroup>

<ProjectReference Include="..\Vitriol.Shared\Vitriol.Shared.csproj" />

<ProjectReference Include="..\Vitriol.Data\Vitriol.Data.csproj" />

<ProjectReference Include="..\Vitriol.Engine\Vitriol.Engine.csproj" />

</ItemGroup>

</Project>